

Estrutura de dados

unopar

Estrutura de dados

Carlos Eduardo Cayres

© 2017 por Editora e Distribuidora Educacional S.A.
Todos os direitos reservados. Nenhuma parte desta publicação poderá ser reproduzida ou transmitida de qualquer modo ou por qualquer outro meio, eletrônico ou mecânico, incluindo fotocópia, gravação ou qualquer outro tipo de sistema de armazenamento e transmissão de informação, sem prévia autorização, por escrito, da Editora e Distribuidora Educacional S.A.

Dados Internacionais de Catalogação na Publicação (CIP)

Cayres, Carlos Eduardo
C385e Estrutura de dados / Carlos Eduardo Cayres. – Londrina: Editora e Distribuidora Educacional S.A., 2017.
252 p.

ISBN 978-85-8482-572-1

1. Estruturas de dados (Computação). I. Título.
CDD 005.1

2017
Editora e Distribuidora Educacional S.A.
Avenida Paris, 675 – Parque Residencial João Piza
CEP: 86041-100 — Londrina — PR
e-mail: editora.educacional@kroton.com.br
Homepage: http://www.kroton.com.br/

Sumário

Tema 1 | Introdução às estruturas de dados 7
Tema 2 | Recursividade 29
Tema 3 | Pilha 47
Tema 4 | Filas 81
Tema 5 | Lista 117
Tema 6 | Árvores binárias 157
Tema 7 | Árvore multidirecional (Árvore B) 185
Tema 8 | Grafos e suas aplicações 217

Convite à leitura

Neste tema, você vai estudar os conceitos básicos sobre Estrutura de Dados, que é o nome dado à forma de organizar dados visando otimizar seu uso. Abordaremos as principais estruturas de dados que podem ser aplicadas na maioria dos problemas com sucesso.

Estrutura de dados é um dos fundamentos da computação empregados em diversas áreas para resolver os problemas mais variados. Para início de conversa, devemos resgatar o conceito de algoritmos, que são estruturas de programação utilizadas para manipular dados, facilitando a compreensão da manipulação das estruturas de dados.

As estruturas de dados estão em constante aprimoramento, bem como os algoritmos e as linguagens de programação, dificultando a escolha da estrutura de dados ideal para a solução de determinado problema. Ainda assim, algumas estruturas consideradas clássicas são sempre uma boa opção, definindo um padrão de estruturas de dados para a solução de desafios.

Introdução às estruturas de dados



POR DENTRO DO TEMA

Computadores são máquinas que manipulam dados e informações. A computação abrange o estudo da forma como as informações são organizadas, manipuladas e utilizadas em um computador.

Ao desenvolver um programa para realizar o processamento de dados, é preciso transcrever de forma que o computador possa compreender e executar tal programa e que o programador também compreenda o que escreveu. As linguagens de programação são códigos escritos em uma linguagem que o programador compreende e que o computador consegue interpretar e executar.

> **Qual é o verdadeiro significado de informação? Por um lado, o conceito de informação na ciência da computação é semelhante aos conceitos de ponto, linha e plano, na geometria: todos eles são termos indefinidos sobre os quais podem ser feitas afirmações, mas eles podem ser explicados em termos de conceitos elementares (TENENBAUM; LANGSAM; AUGENSTEIN, 1995).**

Inteiros Binários e Decimais

O sistema numérico binário é a base do funcionamento dos computadores. O sistema numérico transforma os dados em 0 e 1, e só assim podem ser armazenados na memória. Os dígitos binários são organizados na memória em byte (oito 0 e 1 agrupados, 8 bits), sendo que cada byte é associado a um endereço de memória, o que facilita sua identificação e localização.

Nos sistemas computacionais, os caracteres (letras, números e símbolos) são identificados por um caractere numérico correspondente na tabela ASCII, sendo esse caractere numérico convertido em binário para, posteriormente, ser armazenado na

memória.

Assim, cada variável é associada a uma posição de memória, tendo como característica um nome e um tipo predefinidos. Variáveis podem armazenar valores diferentes ao longo da execução de um programa, mas armazenam um único valor a cada passo da execução.

Tipos de Dados

Na maioria dos problemas resolvidos computacionalmente, os tipos de **dados**, numérico (números inteiros, real etc.), literal (caractere ou string), estão entre o mais comuns.

Dados do Tipo Numérico

Tipos de dados como números inteiros não possuem casas decimais, podendo ser números positivos ou números negativos. Para armazenar um dado numérico do tipo inteiro, são necessários 2 bytes de memória (o espaço para armazenamento pode variar dependendo da linguagem de programação).

Exemplos de dados numéricos inteiros:

1025

-33

78

-25301

Tipo de dados como números reais possuem casas decimais, podendo ser números positivos ou números negativos. Para armazenar um dado numérico real, são necessários 4 bytes de memória (o espaço para armazenamento pode variar dependendo da linguagem de programação).

Exemplos de dados numéricos reais:

13.35

123.51

-21.08

0.0

Dados do Tipo Literal ou Caractere

São tipos de dados formados por um caractere ou por uma cadeia de caracteres justapostos. Os caracteres podem ser letras minúsculas, letras maiúsculas, números

e caracteres especiais. Para armazenar um dado do tipo caractere na memória do computador, é necessário um byte por caractere.

Exemplos de dados literais:

'teste'

'1 + 4'

'exemplos!'

Tipos de Variáveis em C

Na linguagem C, as variáveis devem ser declaradas depois da definição do tipo de variável. Os tipos int (armazenar números inteiros), float (armazenar números reais) e char (armazenar um caractere) são os tipos de dados mais utilizados na linguagem C.

Para armazenar uma cadeia de caractere na linguagem C, deve-se utilizar um vetor de elementos do tipo char.

Exemplos de declarações:

float a, b;

Declaração de uma variável chamada a e outra chamada b para armazenar um número real cada uma.

char sexo;

Declaração de uma variável chamada sexo para armazenar um caractere.

char nome[30];

Declaração de uma variável chamada nome para armazenar trinta caracteres.

Vetor na Linguagem C

Os vetores, também chamados de variáveis compostas homogêneas unidimensionais, apresentam como uma de suas características a capacidade de armazenar vários valores (dados) com uma única referência de nome dado ao vetor, sendo diferenciados pelo índice do vetor.

Para identificar as posições de um vetor na linguagem C, é recomendável iniciar sempre em 0 (zero) e terminar com o valor que indicar o tamanho do vetor (quantidade de posições disponíveis para armazenar dados) menos um.

Declaração de Vetor em C

Na linguagem C, os vetores podem ser identificados na declaração por colchetes depois do nome da variável. O número entre os colchetes indica quantas posições tem o vetor, ou seja, a quantidade de dados que é capaz de armazenar.

Exemplo de vetor:

Vejamos a declaração de um vetor chamado vet com 10 posições de memória, iniciando com índice 0 e indo até 9 (9 é igual ao tamanho do vetor - 1). O tipo int na declaração do vetor indica que poderão ser armazenados tipos de dados numéricos inteiros.

int vet[10];

vet	7	2	5	10	3	21	44	23	4	9
	0	1	2	3	4	5	6	7	8	9

Atribuindo Valores a um Vetor em C

float vet[5] = 3.5;

Significa que vet na posição 5 recebe por atribuição o valor 3.5.

Carregando Valores em um Vetor em C

Para carregar dados em um vetor (ler dados do teclado) e atribuí-los a um vetor, podemos usar o trecho de código a seguir:

```
for (i=0;i<10;i++)
        scanf("%d", &vet[i]);
```

Imprimindo Valores de um Vetor em C

```
for (i=0;i<10;i++)
        printf("%d\n", vet[i]);
```

Exemplos Vetores - Problemas Resolvidos na Linguagem C

Exemplo de um programa em C que carrega um vetor com 10 números inteiros, calcula e mostra dois vetores resultantes contendo os números positivos e os números negativos, respectivamente. Os vetores resultantes poderão ter 10 posições no máximo, sendo que nem todas as posições poderão ser preenchidas.

```
#include <iostream>

#include <stdio.h>
```

```
int main()
{
  int num[10], pos[10], neg[10], i, cont, cont_n, cont_p;
  cont_n = 0;
  cont_p = 0;
  for (i=0;i<10;i++)
  {
       printf("Digite o %d valor: ", i+1);
       scanf("%d", &num[i]);
       if (num[i] >=0)
          {
         pos[cont_p] = num[i];
            cont_p++;
      }
      else
      {
          neg[cont_n] = num[i];
             cont_n++;
      }
  }
  if (cont_n == 0)
     printf("\nVetor de negativos vazio.\n");
  else
  {
     printf("\nValores negativos: \n");
     for (i=0;i<cont_n;i++)
```

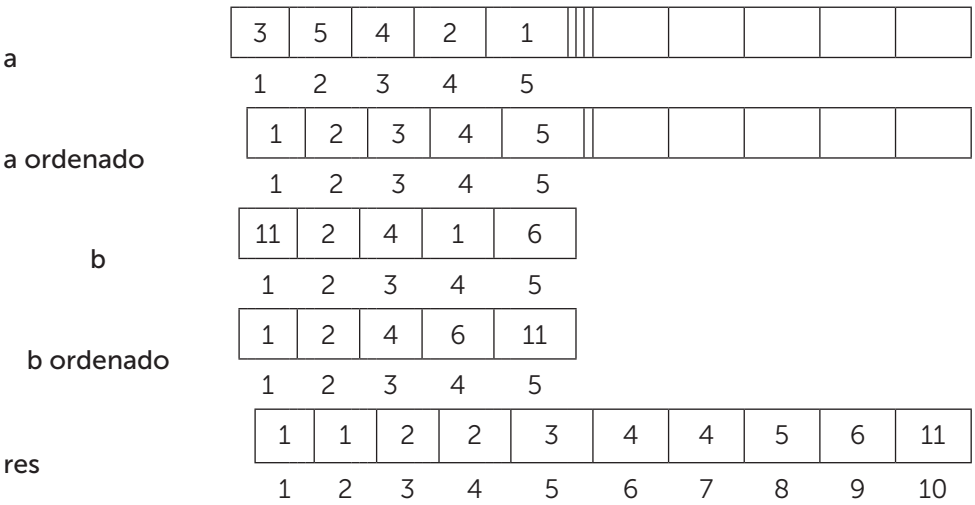
```
            printf("%d \n", neg[i]);
    }
    if (cont_p == 0)
        printf("\nVetor de positivos vazio. \n");
    else
    {
        printf("\nValores positivos: \n");
        for (i=0;i<cont_p;i++)
            printf("%d \n", pos[i]);
    }
    system("PAUSE");
}
```

Exemplo de um programa em C que faz a leitura de um vetor de 10 posições de números inteiros, colocando os números em ordem crescente durante a leitura.

```
#include <iostream>
#include <stdio.h>
int main()
{
  int vet[10], i, j, y, aux;
  for (i=0;i<10;i++)
     { printf("Digite um número: ");
       scanf("%d", &aux);
       j = 0;
       while ((vet[j] < aux) && (j<i))
    j = j + 1;
       for (y=i;y>j;y--)
```

```
            vet[y] = vet[y-1];
        vet[j] = aux;
    }
    printf("\nVetor Ordenado \n");
    for (i=0;i<10;i++)
        printf("%d ", vet[i]);
    system("PAUSE");
}
```

Exemplo de um programa em C que carrega dois vetores com 5 números inteiros cada. Na sequência, ordenar os vetores na ordem crescente. Imprimir um terceiro vetor com 10 posições em ordem crescente, resultante da intercalação dos dois vetores.




```
#include <iostream>
#include <stdio.h>
int main()
{
  int a[5], b[5], res[10], i, j, y, aux;
  for (i=0;i<5;i++)
```

```
{
    printf("Digite o %d° valor do vetor a: ", i+1);
    scanf("%d", &a[i]);
}
for (i=0;i<5;i++)
{
     for (j=0;j<4;j++)
        {
        if (a[j] > a[j+1])
           {
          aux = a[j];
                    a[j] = a[j+1];
                    a[j+1] = aux;
           }
       }
}
for (i=0;i<5;i++)
{
    printf("Digite o %d° valor do vetor b: ", i+1);
    scanf("%d", &b[i]);    }
    for (i=0;i<5;i++)
    {
        for (j=0;j<4;j++)
           {
           if (b[j] > b[j+1])
              {
```

```
            aux = b[j];
            b[j] = b[j+1];
                        b[j+1] = aux;
          }
       }
}
j = 0;
for (i=0;i<5;i++)
{
   res[j] = a[i];
   j++;
   res[j] = b[i];
   j++;
}
for (i=0;i<10;i++)
{
   for (j=0;j<9;j++)
      {
      if (res[j] > res[j+1])
         {
        aux = res[j];
                  res[j] = res[j+1];
                  res[j+1] = aux;
      }
   }
}
```

```
    printf("\nVetor a: \n");
    for (i=0;i<5;i++)
        printf("%d ", a[i]);
    printf("\nVetor b: \n");
    for (i=0;i<5;i++)
        printf("%d ", b[i]);
    printf("\nVetor resultante: \n");
    for (i=0;i<10;i++)
        printf("%d ", res[i]);
    system("PAUSE");
}
```

Matriz na Linguagem C

Matrizes podem ser definidas como um conjunto de variáveis de mesmo tipo, identificadas pelo mesmo nome. Para referenciar as posições de memória de uma matriz, é preciso especificar suas posições dentro desta estrutura.

Na linguagem C, uma matriz pode ser declarada como unidimensional (mais conhecida como vetor), bidimensional e multidimensional.

Embora as matrizes mais utilizadas sejam as bidimensionais (apenas 2 dimensões), alguns compiladores podem trabalhar com até 12 dimensões.

Declarando uma Matriz em C

Vejamos a declaração de uma matriz chamada mat com 3x4 posições de memória, indicando que é uma matriz com 3 linhas e 4 colunas, em que os índices responsáveis por referenciar as posições da matriz devem ser iniciados em 0 e indo até 2 e até 3. O tipo float na declaração da matriz indica que poderão ser armazenados tipos de dados numéricos reais.

```
float mat[3][4];
```

A exemplo dos vetores, os índices da matriz devem começar sempre em 0 (zero). Na declaração apresentada, a variável chamada mat contém 3 linhas (0 a 2) e 4 colunas (0 a 3), capazes de armazenar números reais, como pode ser observado a seguir:

		0	1	2	3
mat	0				
	1				
	2				

Atribuindo Valores a uma Matriz em C

```
mat[2][1]=3
```

O valor 3 será atribuído à posição referente à linha 2 (3ª linha), coluna 1 (2ª coluna) da matriz.

		0	1	2	3
mat	0				
	1				
	2		3		

Carregando Valores em uma Matriz em C

Para carregar dados em uma matriz (ler dados do teclado) e atribuí-los a uma matriz, podemos usar o trecho de código a seguir:

```
for (i=0;i<3;i++)
{
 for (j=0;j<4;j++)
        scanf("%d",&mat[i][j]);
}
```

No exemplo apresentado, observamos uma matriz com 3 linhas, portanto, o comando for mais externo variou de 0 a 2 (para percorrer as 3 linhas da matriz) e o comando for mais interno variou de 0 a 3 (para percorrer as 4 colunas da matriz).

Imprimindo os Dados de uma Matriz em C

Para imprimir (mostrar) os valores da matriz do exemplo anterior, podemos usar o código a seguir:

```
for (i=0;i<3;i++)
{
```

```
for (j=0;j<4;j++)
        printf("%d \n", mat[i][j]);
}
```

Exemplos Matriz - Problemas Resolvidos na Linguagem C

Exemplo de um programa em C que carrega uma matriz 5x3 com a pontuação de 5 ginastas em 3 aparelhos. Listar o número do ginasta (número da linha) e aparelho em que cada ginasta obteve menor nota. Para finalizar, listar a quantidade de ginastas com menor nota no primeiro aparelho, a quantidade de ginastas com menor nota no segundo aparelho e a quantidade de ginastas com menor nota no terceiro aparelho.

```
#include <iostream>
#include <stdio.h>
int main()
{
  float notas[5][3], menor;
  int a1, a2, a3, menor_nota, i, j;
  for (i=0;i<5;i++)
  {
     for (j=0;j<3;j++)
        {
        printf("\nDigite a %dª nota do ginasta %d: ", j+1, i+1);
           scanf("%f", &notas[i][j]);
     }
  }
  a1 = 0, a2 = 0, a3 = 0;
  for (i=0;i<5;i++)
  {
     printf("\nGinasta número: %d", i+1);
```

```
        menor = notas[i][0];
        menor_nota = 0;
        for (j=0;j<3;j++)
            {
            if (notas[i][j] < menor)
                {
                menor = notas[i][j];
                    menor_nota = j;
                }
        }
       printf("\nA menor nota do ginasta %d foi no %dª aparelho.", i+1, menor_nota);
        if (menor_nota == 0)
                a1 = a1 + 1;
        if (menor_nota == 1)
                a2 = a2 + 1;
        if (menor_nota == 2)
                a3 = a3 + 1;
    }
    printf("\nQuantidade de ginastas com menor nota no aparelho 1 = %d.", a1);
    printf("\nQuantidade de ginastas com menor nota no aparelho 2 = %d.", a2);
    printf("\nQuantidade de ginastas com menor nota no aparelho 3 = %d.", a3);
    system("PAUSE");
}
```

Exemplo de um programa em C que carrega uma matriz 3x5 com números inteiros e calcula a soma de cada linha. O resultado da soma deverá ser armazenado em um vetor. Em seguida, o programa deverá multiplicar cada elemento da matriz pela soma da sua linha e imprimir a matriz resultante.

```
#include <iostream>
#include <stdio.h>
int main()
{
     int mat[3][5], soma[3], i, j;

     printf("\nDigite os elementos da matirz: \n");
     for (i=0;i<3;i++)
     {
       for (j=0;j<5;j++)
       {
         printf("\nMat[%d][%d]= ", i, j);
         scanf("%d", &mat[i][j]);
       }
    }
    for (i=0;i<3;i++)
    {
        soma[i] = 0;
        for (j=0;j<5;j++)
            soma[i] = soma[i] + mat[i][j];
    }
    for (i=0;i<3;i++)
    {
        for (j=0;j<5;j++)
            mat[i][j] = mat[i][j] * soma[i];
    }
```

```
    printf("\nMatriz resultante");
    for (i=0;i<3;i++)
    {
        printf("\nLinha %d \n", i);
        for (j=0;j<5;j++)
            printf("%d ", mat[i][j]);
    }
    system("PAUSE");
}
```



ACOMPANHE NA WEB

LEE, Huei Diana; PERES, Fabiana F. F.; MARTINS, Ana Paula. Introdução e Conceitos: Tipos de Dados, Estruturas de Dados e Tipos Abstratos de Dados.

Estruturas de dados é o nome dado à organização de dados de forma coerente e racional, buscando melhorar seu uso. Dependendo de como um conjunto de dados é organizado e de como as operações são efetuadas sobre esses dados, é possível solucionar problemas complexos de forma simples.

Disponível em: <http://www.foz.unioeste.br/~frata/aed/material_didatico_aed/1oBim/Aula2.pdf>. Acesso em: 21/05/2014.

HECK JUNIOR, Vilson. Lógica de Programação, Algoritmos e Estruturas de Dados.

Vários são os modelos de estruturas de dados, sendo alguns bastante clássicos. Ainda assim, novos modelos surgem constantemente, acompanhando a evolução dos algoritmos e das linguagens de programação.

Disponível em: <http://docente.lages.ifsc.edu.br/vilson.junior/MaterialDidatico/ip/IP_01_Logica.pdf>. Acesso em: 21/05/2014.

LIMA, Ricardo Massa F.; SOARES, Sérgio C. B. Aula Extra - Introdução a Estruturas de Dados.

Introdução às Estruturas de Dados: Lista, Pilha, Fila e Árvore, com implementação de um exemplo de lista em Java.

Disponível em: <https://www.youtube.com/watch?v=FI2L2yzeUv0>. Acesso em: 21/05/2014.

Tempo 1:20.



AGORA É A SUA VEZ

Instruções:

Agora, chegou a sua vez de exercitar seu aprendizado. A seguir, você encontrará algumas questões de múltipla escolha e dissertativas. Leia cuidadosamente os enunciados e atente-se para o que está sendo pedido.

Questão 1

Com o intuito de verificar as habilidades de programação que adquiriu até agora, solucione um problema simples utilizando os recursos de programação estudados. Tais conhecimentos são extremamente necessários na sequência dos conteúdos, pois teremos muitos exemplos e exercícios baseados na linguagem C.

Problema proposto: Implemente um programa em C que leia 10 conjuntos de 2 valores, o primeiro representando o número do aluno e o segundo representando sua média final. Encontre o aluno com a maior e com a menor média final. Ao final da execução, mostre o número do aluno com a maior média final junto à sua média e o número do aluno com a menor média final junto à sua média.

Questão 2

No que se refere à declaração de vetores unidimensionais na linguagem C, identifique a alternativa que corresponde à declaração de um vetor de inteiros com espaço de armazenamento para 20 números.

a) int vet[20].

b) float vet [20].

c) char vet[20].

d) int vet[10][10].

e) Nenhuma das alternativas é verdadeira.

Questão 3

Na linguagem C, para um vetor cujo limite mínimo é 0 (zero) e o limite máximo é 29 (vinte e nove), qual é a faixa correspondente entre o limite mínimo e máximo do vetor?

a) 0.

b) 29.

c) 31.

d) 30.

e) Nenhuma das alternativas é verdadeira.

Questão 4

Com base na estrutura de programação da linguagem C, crie um programa que faça a leitura de dois vetores de 5 elementos numéricos cada um e apresente como saída um 3º vetor, resultante da intercalação desses dois vetores. Veja o exemplo a seguir:

vet1	2	4	6	8	10					
vet2	3	5	7	9	11					
vet3	2	3	4	5	6	7	8	9	10	11

Questão 5

Com base na estrutura de programação da linguagem C, crie um programa que faça a leitura de um vetor bidimensional (matriz) 2x2 de elementos numéricos inteiros. Como saída, mostre uma matriz resultante, que será a matriz digitada multiplicada pelo maior valor encontrado na matriz.



FINALIZANDO

Neste tema, você aprendeu conceitos básicos sobre estruturas de dados, que é o nome dado à maneira como os dados são organizados visando otimizar seu uso. Foram abordadas as principais estruturas de dados que geralmente são utilizadas para a solução dos mais diversos problemas.

Na computação, a estrutura de dados é um dos recursos empregados em diversas áreas para resolver os problemas mais variados, simples ou complexos.

É importante observar que seu constante aprimoramento dificulta um pouco a escolha do tipo de estrutura de dados mais recomendado para determinado problema; ainda assim, algumas estruturas clássicas são sempre uma boa opção.

REFERÊNCIAS

ASCENCIO, A. F. G.; CAMPOS, E. A. V. Fundamentos da programação de computadores. 2. ed. São Paulo: Prentice Hall, 2007.

HECK JUNIOR, Vilson. Lógica de Programação, Algoritmos e Estruturas de Dados. Disponível em: <http://docente.lages.ifsc.edu.br/vilson.junior/MaterialDidatico/ip/IP_01_Logica.pdf>. Acesso em: 21/05/2014.

LEE, Huei Diana; PERES, Fabiana F. F.; MARTINS, Ana Paula. Introdução e Conceitos: Tipos de Dados, Estruturas de Dados e Tipos Abstratos de Dados. Disponível em: <http://www.foz.unioeste.br/~frata/aed/material_didatico_aed/1oBim/Aula2.pdf>. Acesso em: 21/05/2014.

LIMA, Ricardo Massa F.; SOARES, Sérgio C. B. Aula Extra - Introdução a Estruturas de Dados. Disponível em: <https://www.youtube.com/watch?v=FI2L2yzeUv0>. Acesso em: 21/05/2014.

TENENBAUM, Aaron M.; LANGSAM, Yedidyah; AUGENSTEIN, Moshe J. Estruturas de dados usando C. São Paulo: Makron Books, 1995.

GLOSSÁRIO

Dados: são características observadas de qualquer coisa (objeto, pessoa, sistema) que possam ser coletadas e armazenadas com ou sem uso de computador. Os dados podem ser coletados e anotados em fichas, por exemplo. Podem consistir em números, palavras, imagens, entre outros. As características de um usuário são dados do mesmo: idade, sexo, CPF, endereço etc. Na computação, dados selecionados e armazenados sem nenhum tipo de tratamento ou transformação são classificados como dados brutos.

Informação: pode-se definir informação como um conjunto organizado de dados. Informação precisa ajuda na tomada de decisões estratégicas e na solução de problemas. Por outro lado, pode-se dizer que informação é um fenômeno que atribui significado ou sentido às coisas.

Estrutura de Dados: na Computação, estrutura de dados é uma forma específica de organização e armazenamento de dados, para que sejam utilizados de forma eficaz. As estruturas de dados e seus algoritmos são muito utilizados na Ciência da Computação em diversas áreas do conhecimento e com as mais diferentes finalidades na solução de problemas computacionais.

Recursividade



POR DENTRO DO TEMA

Introdução ao conceito de Funções e Recursividade em C

Antes de partir para recursividade propriamente dita vamos fazer uma revisão sobre funções em C para que possamos compreender melhor o paradigma da recursividade, já que recursividade está diretamente relacionada ao conceito de função e sua implementação.

Funções em C

Funções são trechos de programa que realizam atividades bem específicas em determinado momento da execução. Todo programa escrito na linguagem de programação C possui, no mínimo, uma função chamada de função principal do programa, a função main, que é responsável pela execução do programa.

Em alguns casos, as funções recebem valores do programa principal para poderem trabalhar. Estes valores são chamados de parâmetros.

Outro elemento importante é o retorno da função, valor que a função produz e manda de volta ao programa principal ou para outra função.

Exemplo 1: Programa em C que receba duas notas, calcule e mostre a média aritmética usando uma função. A função não receberá parâmetros e não retornará nenhum valor ao programa principal.

```
#include <iostream>

#include <stdio.h>

void calc_media();

int main()

{ calc_media(); }

  void calc_media(){
```

```
    float nota1, nota2, media;
    printf( "Digite a primeira nota: ");
    scanf("%d", &nota1);
    printf( "Digite a segunda nota: ");
    scanf("%d", &nota2);
    media = (nota1+ nota2)/2;
    printf( "Média=  %d", media);
    system("PAUSE");
}
```

Ao desenvolver programas em C utilizando funções definidas pelo usuário (como no exemplo anterior), estas podem ser escritas antes ou depois da função main. Se for escrita depois da função main, é preciso adicionar uma instrução para que o compilador possa interpretar a função quando estiver percorrendo linearmente todas as linhas do programa. Informações como, a quantidade e o tipo de parâmetros, se tem ou não retorno de valor e qual o seu tipo. Tal instrução deve ser acrescentada antes da função main e é chamada de protótipo da função.

Passagem de parâmetros

Cada função pode receber vários valores chamados de parâmetros, e pode devolver com retorno um valor. Dessa forma, quando se especifica uma função deve-se deixar claro qual será o tipo de retorno e quais os parâmetros necessários para que à execução da função.

Exemplo 1:

```
int soma_num (int n1, int n1)
{
int soma;
        soma = n1 + n2;
        return soma;
}
```

No exemplo 1, observa-se que foi realizada a soma de dois valores inteiros (n1 e n2) que foram passados como parâmetros para a função. O resultado desta

soma, armazenado na variável soma que em seguida é devolvido ao ponto em que a função foi chamada durante a execução do programa. Isto só foi possível porque a função estava preparada para tratar estes valores, conforme especificado na primeira linha da função int soma_num (int n1, int n1).

Exemplo 2:

```
float dividir_num(int dividendo, int dividor)
{
float res;
    res = dividendo / divisor;
    return res;
}
```

No exemplo 2 foi realizada uma operação de divisão. Pode-se observar que a função recebeu dois valores inteiros para proceder a operação. É sabido que o resultado de uma divisão pode gerar valores reais, por isso, o resultado teve que ser armazenado em uma variável do tipo float, que após receber o resultado da divisão foi devolvido ao ponto em que a função foi chamada na execução do programa. Como o resultado da divisão pode gerar um número real a função dividir_num foi definida como sendo float.

Exemplo 3:

```
void multiplicar_num (float multiplicando, float multiplicador)
{
float res;
    res = multiplicando * multiplicador;
    printf( "\nO resultado da multiplicação é= %d", res);
    system("PAUSE");
}
```

No exemplo 3 podemos observar outra variação da utilização da função. Neste caso a função recebeu dois parâmetros do tipo float para calcular a multiplicação. O resultado da operação de multiplicação foi mostrado dentro da própria função, sendo assim, a função não tem retorno e foi definida como void.

Passagem de parâmetros por valor

Passagem de parâmetros por valor significa que, para a execução da função, serão geradas cópias dos valores de cada um dos parâmetros. Como exemplo, observe o programa abaixo:

```
1-  #include <iostream>
2-  #include <stdio.h>
3-  int soma_dobro(int a, int b);
4-  int main()
5-  { int x, y, res;
6-   printf("\nDigite o primeiro número: ");
7-   scanf("%d", &x);
8-   printf("\nDigite o segundo número: ");
9-   scanf("%d", &y);
10- res = soma_dobro(x,y);
11- printf("\nA soma do dobro dos números %d e %d é: %d", x, y, res);
12- system("PAUSE"); }
13- int soma_dobro(int a, int b)
14- {  int soma;
15- a = 2*a;
16- b= 2*b;
17- soma = a + b;
18- return soma;
19- }
```

Estamos supondo que os valores armazenados nas variáveis x e y, através da execução das linhas 7 e 9 do programa anterior, foram respectivamente, 5 e 3. Quando a linha 10 é executada, estes valores são copiados para as variáveis a e b (pertencentes à função soma_dobro). Depois disso, os valores de a e b são multiplicados por 2 e, depois, é realizada a soma. O resultado desta soma é devolvido à função main através da execução da linha 18, onde o valor calculado

(16) recai sobre a variável res. No momento em que a função soma_dobro chega ao fim, as variáveis a, b e soma são destruídas e, portanto, as alterações realizadas através das multiplicações por 2 são perdidas.

Passagem de parâmetros por referência

Na passagem de parâmetros por referência os parâmetros passados para uma função fazem referência a endereços de memória alocados para variáveis. Isso significa que toda vez que for preciso acessar determinado valor de uma variável, só poderá ser feito referenciando seu endereço.

```
1-  #include <iostream>
2-  #include <stdio.h>
3-  int soma_dobro(int *a, int *b);
4-  int main()
5-  { int x, y, res;
6-    printf("\nDigite o primeiro número: ");
7-    scanf("%d", &x);
8-    printf("\nDigite o segundo número: ");
9-    scanf"%d", &y);
10-  res = soma_dobro(&x,&y);
11-  printf("\nA soma dos números %d e %d é: %d", x, y, res);
12-  system("PAUSE"); }
13-  int soma_dobro(int *a, int *b)
14-  {  int soma;
15-     *a = 2*(*a);
16-     *b= 2*(*b);
17-     soma = *a + *b;
18-     return soma; }
```

Nas linhas 7 e 9 são lidos, respectivamente, os valores para as variáveis x e y (como exemplo, estamos supondo que foram digitados os valores 5 e 3). Entretanto, quando a função soma_dobro é chamada, na linha 10, são passados como

parâmetros para a função os endereços de memória ocupados pelas variáveis x e y (isto é feito através do operador & - que obtém o endereço de memória de uma variável), ou seja, conforme nosso exemplo, os valores 800 (endereço ocupado por x) e 300 (endereço ocupado por y). Dessa forma, os valores que recaem sobre as variáveis a e b (da função) são, respectivamente, 800 e 300 (isto é correto, uma vez que a e b são ponteiros para int).

Nas linhas 15 e 16, os valores 5 e 3 são multiplicados por 2. Neste momento ocorre a "referência" aos endereços de memória 800 e 300, para que sejam obtidos os valores iniciais e, após a realização das multiplicações, sejam alterados os valores alterados e, assim, no endereço 800 passamos a ter o valor 10 e no endereço 300 passamos a ter o valor 6. Na linha 17, é realizada a soma dos valores que estão nos endereços especificados por a e b (que já foram multiplicados por 2).

Finalmente, na linha 18, o resultado da soma é devolvido à função main, recaindo sobre a variável res e encerrando a função soma_dobro. Quando a função soma_dobro chega ao fim, as variáveis a, b e res são destruídas, entretanto, as alterações decorrentes das multiplicações feitas são mantidas, pois não geraram duplicatas de valores, mas sim, a todo momento, fizeram referência a endereços de memória que estavam fora da área destinada à função.

Em muitas linguagens de programação, vetores e matrizes são passados para funções como os demais tipos de dados, por valor e por referência. Entretanto, quando a linguagem C é utilizada, pode-se apenas passá-los por referência, ou seja, a função recebe apenas o endereço da primeira posição do vetor ou da matriz.

Exemplo 1:

```
int vet[10];

exemplo(&vet[0]); //o endereço de memória ocupado pela 1ª posição do vetor é passado para a função exemplo.

exemplo(vet); //quando se utiliza apenas o nome do vetor (sem colchetes e índice), significa que também estamos pegando o endereço de memória ocupado pela 1ª posição do vetor.

void exemplo(int v[]) ou void exemplo(int *x)

{

//linhas de comando

}
```

Exemplo 2:

```
float mat[50][50];

exemplo2 (&mat[0][0]);

exemplo2 (mat);

void exemplo2 (float a[][50]);

{

//linhas de comando

}
```

Recursividade em C

Um objeto é denominado **recursivo** se ele é parcialmente definido em termos dele mesmo. A recursividade é encontrada, principalmente, na matemática, porém, está presente em várias situações do nosso cotidiano.

Na programação, a recursividade segue o mesmo conceito da matemática: dividir o problema em instâncias menores (sem alterar as características do problema) para facilitar a obtenção da resposta. Uma função é considerada recursiva se ela contém uma chamada a si mesma. Deve-se considerar o momento em que a recursividade termina, pois, caso isto não seja feito, o programa ficará em execução infinitamente (loop infinito) até esgotar os recursos disponíveis no computador (que irá travar).

Como exemplo de problema que pode ser resolvido recursivamente podemos citar o cálculo de um fatorial. Para realizar o cálculo do fatorial de um número (N) algumas regras precisam ser seguidas:

se N = 0 ou N = 1 então o fatorial(N) = 1;

se N > 1 então fatorial(N) = fatorial(N) * fatorial(N-1).

ou seja, o valor do fatorial do número 4 pode ser obtido assim:

Fatorial(4) = 4 * Fatorial(3)

Fatorial(3) = 3 * Fatorial(2)

Fatorial(2) = 2 * Fatorial(1)

Fatorial(1)=1

Observe que o fatorial de 4 foi obtido através do cálculo do fatorial de 3, que foi obtido através do cálculo do fatorial de 2 e assim por diante, até chegar em 1 (pois o valor do fatorial de 1 é obtido diretamente).

A função fatorial

> Conforme Tenenbaum, na matemática, vários objetos são definidos apresentando-se um processo que os produz. Por exemplo π, é definido como a razão entre a circunferência de um círculo e seu diâmetro. Isso equivale ao seguinte conjunto de instruções: obter a circunferência de um círculo e seu diâmetro, dividir o primeiro pelo último e chamar o resultado de π. Evidentemente, o processo especificado precisa terminar com um resultado definido. (TENENBAUM; LANGSAM; AUGENSTEINl, 1995, p. 132).
> A função fatorial é considerada um exemplo de definição especificada por um processo que desempenha um papel importante na matemática e na estatística. O fatorial de um número inteiro positivo n, é definido como o produto de todos os inteiros entre n e 1. Por exemplo, o fatorial de 4 é igual a 4 * 3 * 2 * 1 = 24. (TENENBAUM; LANGSAM; AUGENSTEINl, 1995, p. 133).

Temos também a definição do fatorial de 0 (zero) que é 1. O ponto de exclamação (!) é o símbolo utilizado para indicar função fatorial. A definição da função fatorial pode ser escrita dessa forma:

n ! = 1 se n == 0

n! = n * (n - 1) * (n - 2) * ...* 1 se n > 0

Na representação acima os três pontos significam a abreviatura para a sequência dos números existentes entre n - 3 e 2 multiplicados. Se não quisermos abreviar a definição de n!, temos que escrever uma fórmula para n! representando separadamente cada valor de n!:

0! = 1

l! = 1

2! = 2 * 1

3! = 3 * 2 * 1

4 ! =4*3*2* 1

> Evidentemente, não esperamos listar uma fórmula para o fatorial de cada inteiro. Para evitar qualquer abreviatura e um

> conjunto infinito de definições, e ainda assim definir a função precisamente, podemos apresentar um algoritmo que aceite um inteiro n e retorne o valor de n!. (TENENBAUM; LANGSAM; AUGENSTEINl, 1995, p. 133).

```
prod =1;

for (x = n; x > 0; x--)

prod *= x;

return(prod);
```

O trecho de programa acima precisa da repetição de determinado processo até que uma condição seja satisfeita, tal programa é classificado como **iterativo**.

> Um algoritmo pode ser considerado um programa para uma máquina "ideal", sem quaisquer limitações práticas de um computador real e, portanto, pode ser usado para definir uma função matemática. Entretanto, uma função em C não serve como definição matemática da função fatorial por causa de limitações como a precisão e o tamanho finito de uma máquina real. (TENENBAUM; LANGSAM; AUGENSTEINl, 1995, p. 134).

O programa abaixo demonstra um estrutura recursiva para calcular o fatorial de um número inteiro qualquer definido como n!.

```
1 if (n == 0)

2     fat =1 ;

3 else {

4     x = n - 1;

5     ache o valor de x!. Chame-o de y;

6     fat = n * y;

7 } /* fim else */
```

> Evidentemente, a chave do algoritmo é a linha 5, onde somos solicitados a "achar o valor de x!". Isto exige a reexecução do algoritmo com a entrada x porque o método para computar a função fatorial é o próprio algoritmo. Para verificar se o algoritmo ocasionalmente pára, observe que no início da linha 5 x é igual a n - 1. Cada vez que o algoritmo for executado, sua entrada será um a menos que na vez anterior, para que (como a entrada n original era um inteiro não negativo) 0 seja eventualmente a entrada do algoritmo. Nesse ponto, o algoritmo apenas retorna 1. Esse valor é retornado à linha 5, que solicitou a avaliação de 0! A multiplicação de y (igual a 1) por n (igual a 1) é executada e o resultado é retornado. Essa sequência de multiplicações e retornos prossegue até que o n! original seja avaliado. (TENENBAUM; LANGSAM; AUGENSTEINl, 1995, p. 135).

Na avaliação da função fatorial, fica evidente que o método interativo é mais simples e rápido. O método recursivo foi mostrado sem a intenção de apresentar uma forma mais eficiente de resolver o problema do fatorial, mas sim, como um exemplo simples para entendimento da recursividade.

Empilhamento das Chamadas

Antes do programa desviar o curso da execução para uma função, o endereço de retorno é armazenado na pilha de execução. Quando a função chegar ao fim o programa retornará exatamente para este endereço que, então será apagado da pilha de execução.

Importante: nem todos os problemas podem ser resolvidos de forma recursiva, cabendo apenas a solução interativa. Entretanto, todos os programas recursivos podem ser resolvidos iterativamente.

ACOMPANHE NA WEB

Recursividade em C

Quando um método recursivo é chamado para resolver algum problema, na verdade, o método é capaz de resolver apenas o(s) caso(s) mais simples(s) ou o(s) caso(s) básico(s). Se a chamada do método for de um método básico, o método retorna o caso base. Se o método for chamado com um problema mais complexo,

em geral o método divide o programa em duas partes conceituais – uma parte que o método sabe como fazer e a outra que o método não sabe.

Disponível em: <http://linguagemc.com.br/recursividade-em-c>. Acesso em: 18 ago. 2014.

Recursividade em Linguagem C

A função recursiva é um recurso que determinadas linguagens de programação tem que permite executá-las a si mesmas, ou seja, dentro de um código, tem a capacidade de chamar um "subcódigo" para ser executado.

Disponível em: <http://pt.slideshare.net/leoolima/recursividade-em-linguagem-c>. Acesso em: 18 ago. 2014.

Estrutura de Dados I - Aula1 – Parte 1 Recursividade

A ideia de recursividade no traz a noção de ações que se repetem diversas vezes, muitas vezes com objetivos bem definidos.

Disponível em: <https://www.youtube.com/watch?v=iOKlr2zWSc8>. Acesso em: 18 ago. 2014.

Tempo: 10:47



AGORA É A SUA VEZ

Instruções:

Agora, chegou a sua vez de exercitar seu aprendizado. A seguir, você encontrará algumas questões de múltipla escolha e dissertativas. Leia cuidadosamente os enunciados e atente-se para o que está sendo pedido.

Questão 1

Com o intuito de verificar as habilidades de programação adquiridas até agora, a atividade proposta é a solução de um problema simples utilizando os recursos de programação estudados. Tais conhecimentos são extremamente necessários na sequência dos conteúdos, pois teremos muitos exemplos e exercícios baseados na linguagem C.

Problema proposto: Implementar um programa em C que receba o número de peças vendidas por cada vendedor de uma determinada loja de autopeças. O

número de peças vendidas deverá ser armazenado em um vetor. Deve-se receber e armazenar em um segundo vetor o valor da cada peça vendida. Na loja trabalha atualmente 10 vendedores e cada vendedor é responsável por vender apenas um tipo de peça. Como resultado o programa deverá apresentar a quantidade total de peças vendidas pelo 10 vendedores, além de calcular e mostrar por vendedor o valor total das vendas, isto é, a número de peças * o valor de cada peça.

Questão 2

Observe o código abaixo e classifique o algoritmo utilizado com relação a estrutura de programação:

```
prod =1;

for (x = n; x > 0; x--)

prod *= x;

return(prod);
```

a) Recursivo.

b) Iterativo.

c) Retroativo.

d) Analítico incremental.

e) Nenhuma das alternativas é verdadeira.

Questão 3

Qual a saída esperada após a execução do programa recursivo abaixo:

```
long potencia(int n, int p) {
    if ( p == 0 ) {
       return 1;
    }
    else
    {
       if(p != 1)
        n*= potencia (n, p-1);
```

```
        return n;
    }
}

int main(){
    printf("Potência= %d \n", potencia (2, 3));
    system("PAUSE");
    return 0;
}
```

a) 6.

b) 5.

c) 8.

d) 9.

e) Nenhuma das alternativas é verdadeira.

Questão 4

Baseado na estrutura de programação da linguagem C, crie um programa recursivo para calcular a soma dos n primeiros inteiros positivos, sendo n um valor fornecido pelo usuário. Como saída deverá ser mostrado o valor da soma.

Questão 5

Baseado na estrutura de programação da linguagem C, criar um programa recursivo que calcule o número Fibonacci de um número positivo inteiro definido no programa principal. Como saída deverá ser mostrado o número Fibonacci.



FINALIZANDO

Neste tema você aprendeu conceitos e exemplos práticos sobre recursividade baseada nas estruturas de programação recursivas disponíveis na linguagem C.

A linguagem C é uma das mais poderosas linguagens de programação, ela tem recursos para implementação de problemas complexos clássicos da ciência da Computação. Um desses recursos é justamente a recursividade, que permite a solução de problemas complexos de forma simples.

Além disso, foi abordada a definição de algoritmo recursivo e pode-se verificar que uma importante exigência para que um algoritmo recursivo esteja correto é que ele não pode gerar uma sequência infinita de chamadas a si mesmo, pois se gerar tal sequência pode nunca terminar sua execução.



REFERÊNCIAS

CASAVELLA, Eduardo. Recursividade em C. Disponível em: <http://linguagemc.com.br/recursividade-em-c/>. Acesso em: 21 maio 2014.

LIMA, Leonardo. Recursividade em linguagem C. Disponível em: <http://pt.slideshare.net/leoolima/recursividade-em-linguagem-c>. Acesso em: 21 maio 2014.

TENENBAUM, Aaron M. LANGSAM, Yedidyah. AUGENSTEIN, Moshe J. Estruturas de dados usando C. São Paulo: Makron Books, 1995.



GLOSSÁRIO

Estrutura de Dados: na Computação, estrutura de dados é uma forma específica de organização e armazenamento de dados para que sejam utilizados de forma eficaz. As estruturas de dados e seus algoritmos são muito utilizados na Ciência da Computação em diversas áreas do conhecimento e com as mais diferentes finalidades na solução de problemas computacionais.

Recursivo: a recursão é o processo pelo qual um dos passos do procedimento em questão envolve a repetição completa deste mesmo procedimento tantas vezes quanta forem necessárias para solução de um problema.

Iterativo: processo que é executado diversas vezes para se chegar a um resultado e a cada vez gera um resultado parcial que será usado na próxima execução. Repetido muitas vezes.

Pilha



POR DENTRO DO TEMA

A pilha é uma **estrutura de dados** capaz de representar conjuntos de dados organizados em ordem linear. Quando se utilizam vetores nas representações dessas estruturas, são usados endereços contíguos de memória do computador, e a ordem linear é indicada pelos índices dos vetores, sendo que sua implementação em algumas situações pode exigir maior esforço computacional. Tais representações denominam-se pilhas estáticas. Entretanto, quando as estruturas de dados do tipo pilha são representadas por estruturas que contêm o dado e também um ponteiro para o próximo elemento, têm-se elementos encadeados, denominados pilhas dinâmicas.

Quando um elemento de uma pilha contém apenas um tipo de dado, como um número, denomina-se pilha **homogênea**. Quando um elemento de uma pilha contém um dado composto, como o nome e o salário de um funcionário, denomina-se pilha **heterogênea**.

Uma característica importante da estrutura de dados pilha é que o último elemento inserido será o primeiro a ser removido, sendo considerada do tipo LIFO (Last In, First Out).

Na pilha, cada elemento pode armazenar um ou vários dados (estrutura homogênea ou heterogênea, respectivamente) e um ponteiro para o próximo elemento, permitindo o encadeamento e sempre mantendo a estrutura linear.

As operações de inserir na pilha, consultar toda a pilha, remover e esvaziar toda a pilha são as manipulações básicas da estrutura utilizadas na maioria dos problemas.

Qualquer estrutura desse tipo possui um ponteiro denominado TOPO, no qual todas as operações de inserção e remoção acontecem. Assim, as operações ocorrem sempre na mesma extremidade da estrutura.

Inserção e Remoção na Pilha

A operação de inserção e remoção na pilha sempre realiza operações básicas, como a de atribuição, para atualizar o topo da pilha. Logo, são operações de tempo

constante e gastam tempo O(1).

Já a operação de consultar toda a pilha percorre todos os elementos armazenados nela. Considerando que uma pilha contém n elementos, o tempo de execução será O(n).

A operação de esvaziamento da pilha consiste em remover todos os elementos dela. O tempo gasto nessa operação depende da linguagem de programação que está sendo utilizada. Na linguagem C, é necessário desalocar cada um dos elementos da pilha, gastando tempo proporcional ao tamanho dela, ou seja, O(n).

Exemplos

Pilha Estática e Homogênea

São estruturas que têm seu tamanho predefinido, o qual não pode ser alterado durante a execução do programa, e só manipulam um tipo de dado. Como exemplo, a variável do tipo inteiro numero.

```
#include<iostream>
#include<stdio.h>
int main()
{
   int numero[5], op, topo=0, i;
   do
   {
                  system("CLS");
        printf("\n\tMENU");
         printf("\n 1.Inserir número (empilhar)");
         printf("\n 2.Consultar topo");
         printf("\n 3.Consultar toda a pilha");
         printf("\n 4.Excluir (desempilhar)");
         printf("\n 5.Esvaziar a pilha");
         printf("\n 6.Sair");
         printf("\n Digite sua opção: ");
```

```
   scanf("%d", &op);
   if (op<1 || op>6)
      printf("\n Opção inválida.\n");
if (op==1)
{
   if (topo==5)
   printf("\n Pilha cheia.\n");
   else
         {
            printf("\n Digite o n£mero a ser inserido: ");
            scanf("%d", &numero[topo]);
            topo++;
            printf("\n N£mero inserido.\n");
         }
}
if (op==2)
{
   if (topo==0)
   printf("\n Pilha vazia.\n");
   else
   printf("\n Topo: %d ", numero[topo-1]);
}
if (op==3)
{
   if (topo==0)
   printf("\n Pilha vazia.\n");
```

```
   else
          {
            printf("\n N£meros empilhados:");

            for (i=topo-1;i>=0;i--)
                  {
                printf("  %d", numero[i]);
     }
          }
}
if (op==4)
{
   if (topo==0)
   printf("\n Pilha vazia.\n");
   else
          {
            topo--;
            printf("\n Número desempilhado.\n");
          }
}
if (op==5)
{
    if (topo==0)
      printf("\n Pilha vazia.\n");
    else
          {
```

```
                    topo=0;
                    printf("\n Pilha esvaziada.\n");
                }
        }
        system("PAUSE");
      }while(op!=6);
}
```

Pilha Estática e Heterogênea

São estruturas que têm seu tamanho predefinido, o qual não pode ser alterado durante a execução do programa, e manipulam mais de um tipo de dado. Como exemplo, as variáveis do tipo inteiro codigo e do tipo real salario.

```
#include<iostream>
#include<stdio.h>
int main()
{
   struct
   {
      int codigo;
      float sal;
   } pilha[5];
   int op,topo=0,i;
   do
   {
      system("CLS");
      printf("\n\tMENU");
      printf("\n 1.Cadastrar (empilhar)");
      printf("\n 2.Consultar topo");
```

```
printf("\n 3.Consultar toda a pilha");
printf("\n 4.Excluir (desempilhar)");
printf("\n 5.Esvaziar a pilha");
printf("\n 6.Sair");
printf("\n Digite sua opção: ");
scanf("%d", &op);
if (op<1 || op>6)
   printf("\n Opção inválida.\n");
if (op==1)
{
                    if (topo==5)
                    printf("\n Pilha cheia.\n");
                   else
                   {
                              printf("\n Digite o código a ser empilhado: ");
                              scanf("%d", &pilha[topo].codigo);
                              printf("\n Digite o salário: ");
                              scanf("%f", &pilha[topo].sal);
                              topo++;
                              printf("\n Dados empilhados.\n");
                   }
}
if (op==2)
{
                    if (topo==0)
                              printf("\n Pilha vazia.\n");
```

```
                    else
                    {
                        printf("\n código do topo : %d", pilha[topo-1].
codigo);
                        printf("\n Salário do topo: %4.2f", pilha[topo-1].sal);
                    }
            }
            if (op==3)
            {
                     if (topo==0)
                        printf("\n Pilha vazia.\n");
                    else
                    {
                        printf("\n Dados empilhados:");
                        for (i=topo-1;i>=0;i--)
                         {
                         printf("\nCódigo:  %d", pilha[i].codigo);
                         printf("\nSalário: %4.2f", pilha[i].sal);
                         }
                    }
            }
            if (op==4)
            {
                     if (topo==0)
                        printf("\n Pilha vazia.\n");
                     else
```

```
                {
                        topo--;
                        printf("\n Código desempilhado.\n");
                }
        }
        if (op==5)
        {
                if (topo==0)
                        printf("\n Pilha vazia.\n");
                else
                        {
                                topo=0;
                                printf("\nPilha esvaziada.\n");
                        }
        }
        system("PAUSE");
       }while(op!=6);
}
```

Pilha Dinâmica e Homogênea

São estruturas que utilizam ponteiros para indexar endereços de variáveis, permitindo a inserção dinâmica de dados durante a execução do programa, e manipulam apenas um tipo de dado. Como exemplo, a variável do tipo inteiro num.

```
#include<iostream>
#include<stdio.h>
int main()
{
```

```
struct pilha
{
        int num;
        pilha *prox;
   };
   pilha *topo,*aux;
   int op;
   topo=NULL;
   do
   {
    system("CLS");
    printf("\nMENU");
    printf("\n1 - Inserir");
    printf("\n2 - Consultar topo");
    printf("\n3 - Consultar pilha");
    printf("\n4 - Excluir");
    printf("\n5 - Esvaziar pilha");
    printf("\n6 - Sair");
    printf("\nDigite sua opção: ");
    scanf("%d", &op);
    if (op < 1 || op > 6)
    printf("\nOpção inválida.\n");
    if (op == 1)
            {
                    aux=new(pilha);
                    printf("\nDigite o número: ");
```

```
            scanf("%d", &aux->num);
            aux->prox=topo;
            topo=aux;
            printf("\nNúmero inserido.\n");
}
if (op == 2)
{
            if(topo==NULL)
             printf("\nPilha vazia.\n");
            else
            {
             printf("\nNúmero do topo: ");
             printf("%d", topo->num);
            }
}
if (op == 3)
{
            if(topo==NULL)
             printf("\nPilha vazia.\n");
            else
            {
             aux=topo;
             printf("\nToda a pilha: ");
             while(aux!=NULL)
             {
                        printf(" %d", aux->num);
```

```
                    aux=aux->prox;
                }
                }
}
if (op == 4)
{
                if(topo==NULL)
                 printf("\nPilha vazia.\n");
                else
                {
                 aux=topo;
                 topo=topo->prox;
                 delete(aux);
                 printf("\nNúmero do topo excluido.\n");
                }
}
if (op == 5)
{
                if(topo==NULL)
                 printf("\nPilha vazia.\n");
                else
                {
                 while(topo!=NULL)
                 {
                        aux=topo;
                        topo=topo->prox;
```

```
                    delete(aux);
                }
                printf("\nPilha esvaziada.\n");
            }
        }
        system("PAUSE");
    }
    while(op!=6);
}
```

Pilha Dinâmica e Heterogênea

São estruturas que utilizam ponteiros para indexar endereços de variáveis, permitindo a inserção dinâmica de dados durante a execução do programa, e manipulam mais de um tipo de dado. Como exemplo, as variáveis do tipo inteiro codigo e do tipo real salario.

```
#include<iostream>
#include<stdio.h>
int main()
{
  struct pilha
  {
      int codigo;
      float salario;
      pilha *prox;
  };
  pilha *topo,*aux;
  int op;
  topo=NULL;
```

```
do
{
      system("CLS");
      printf("\nMENU");
      printf("\n1 - Inserir");
      printf("\n2 - Consultar topo");
      printf("\n3 - Consultar pilha");
      printf("\n4 - Excluir");
      printf("\n5 - Esvaziar pilha");
      printf("\n6 - Sair");
      printf("\nDigite sua op‡ao: ");
      scanf("%d", &op);
      if (op < 1 || op > 6)
         printf("\nOp‡ao inv lida");
      if (op == 1)
      {
                        aux=new(pilha);
                        printf("\nDigite o código a inserir: ");
                        scanf("%d", &aux->codigo);
                        printf("\nDigite o salário: ");
                        scanf("%f", &aux->salario);
                        aux->prox=topo;
                        topo=aux;
                        printf("\Dados inseridos.\n");
            }
            if (op == 2)
```

```
    {
            if(topo==NULL)
 printf("\nPilha vazia.\n");
            else
            {
             printf("\nInformações do topo\n");
             printf("\Código: %d", topo->codigo);
             printf("\nSalario: %4.2f", topo->salario);
            }
    }
    if (op == 3)
    {
            if(topo==NULL)
printf("\nPilha vazia.\n");
            else
            {
                    aux=topo;
                    printf("\nToda a pilha\n");
             while(aux!=NULL)
             {
                    printf("\nCódigo: %d", aux->codigo);
                    printf("\nSalario: %4.2f\n", aux->salario);
                    aux=aux->prox;
             }
            }
    }
```

```
if (op == 4)
{
        if(topo==NULL)
       printf("\nPilha vazia.\n");
        else
        {
                aux=topo;
                topo=topo->prox;
                delete(aux);
                printf("\nInformações do topo excluídas.\n");
        }
}
if (op == 5)
{
        if(topo==NULL)
                printf("\nPilha vazia.\n");
        else
        {
                while(topo!=NULL)
                {
                        aux=topo;
                         topo=topo->prox;
                        delete(aux);
                }
                printf("\nPilha esvaziada.\n");
        }
```

```
                }
          system("PAUSE");
       }while(op!=6);
}
```

Exemplo de Controle de Estoque Utilizando Pilha Dinâmica Heterogênea

```
#include <iostream>
#include <stdio.h>
#include <dos.h>
int main()
{
   struct pilha
   {
        int cod_op, cod_prod, qtde, dia, mes, ano;
        pilha *posi;
   };
   pilha *novo, *topo, *aux;
   //capturando a data do sistema
   std::time_t tt = std::time(0);
   std::tm ttm = *std::localtime(&tt);
   int mday = ttm.tm_mday;
   int mmon = ttm.tm_mon + 1;
   int myear = ttm.tm_year + 1900;

   int op, dia, mes, ano, prod, cod, qtd, estoque;
   topo = NULL;
   do
```

```
{
   system("CLS");
   printf("\nMenu ");
   printf("\n1 - Cadastrar operação");
   printf("\n2 - Consultar o estoque de um produto");
   printf("\n3 - Consultar as operações a partir de uma data");
   printf("\n4 - Consultar total de produtos em estoque");
   printf("\n5 - Consultar total de perdas");
   printf("\n6 - Retirar a última operação cadastrada");
   printf("\n7 - Esvaziar estoque");
   printf("\n8 - Sair");
   printf("\nDigite sua opção..: ");
   scanf("%d", &op);
   if (op < 1 || op > 8)
     printf("\nOpção inválida.\n");
   if (op == 1)
  {
     printf("\n1 - Compra");
     printf("\n2 - Venda");
     printf("\n3 - Perda");
     printf("\n4 - Devolução");
     printf("\nDigite o tipo de operação: ");
     scanf("%d", &cod);
     while (cod<1 || cod>4)
                                  {
                                          printf("\nOperação inválida.\n");
```

```
                        printf("\n\nDigite novamente: ");
                        scanf("%d", &cod);
                }
printf("\nDigite o c¢digo do produto: ");
scanf("%d", &prod);
printf("\nDigite a quantidade do produto: ");
scanf("%d", &qtd);
while (qtd<=0)
                {
                        printf("\nQuantidade inválida.\n");
                        printf("\nDigite nova quantidade: ");
                        scanf("%d", &qtd);
                }
if (topo==NULL) // pilha vazia, primeira inclusão
        {
                if (cod==2 || cod==3)
                printf("\nEstoque vazio, impossível cadastrar essa operação.\n");
                else
                {
                        novo=new (pilha);
                        novo->dia=mday;
                        novo->mes=mmon;
                        novo->ano=myear;
                        novo->cod_op=cod;
                        novo->cod_prod=prod;
                        novo->qtde=qtd;
```

```
                    novo->posi=topo;
                    topo=novo;
                    printf("\nOperação cadastrada.\n");
                }
            }
        else // já  existe algum produto cadastrado, verificar estoque
            {
              if (cod==2 || cod==3)
                {
                 estoque=0;
                 aux=topo;
                 while (aux!=NULL)
                        {
                            if (aux->cod_prod==prod)
                            {
                             if (aux->cod_op==1 || aux->cod_
op==4)
                                estoque=estoque+aux-
>qtde;
                                else
                                estoque=estoque-aux-
>qtde;
                            }
                            aux=aux->posi;
                        }
                        if (qtd>estoque)
                        printf("\nEstoque insuficiente.\n");
```

```
                    else
                    {
                        novo=new (pilha);
                novo->dia=mday;
                novo->mes=mmon;
                novo->ano=myear;
                        novo->cod_op=cod;
                        novo->cod_prod=prod;
                        novo->qtde=qtd;
                        novo->posi=topo;
                        topo=novo;
                        printf("\nOperação
cadastrada.\n");
                    }
            }
        else
            {
                    novo=new (pilha);
                novo->dia=mday;
                novo->mes=mmon;
                novo->ano=myear;
                    novo->cod_op=cod;
                    novo->cod_prod=prod;
                    novo->qtde=qtd;
                    novo->posi=topo;
                    topo=novo;
```

```
                                        printf("\nOperação cadastrada.\n");
                                }
                        }
                }
                if (op == 2) // consultar o estoque de um determinado produto
                 {
                  if (topo == NULL)
                  printf("\nEstoque vazio.\n");
                else
                {
                        estoque=0;
                        printf("\nDigite o código do produto que deseja consultar o
estoque: ");
                        std::cin>>cod;
                        aux=topo;
                        while (aux!=NULL)
                        {
                          if (aux->cod_prod==cod)
                                {
                                        if (aux->cod_op==1 || aux->cod_op==4)
                                        estoque=estoque+aux->qtde;
                                        else
                                        estoque=estoque-aux->qtde;
                                }
                        aux=aux->posi;
                        }
```

```
            printf("\nQuantidade em estoque: %d", estoque);


  }
}
if (op == 3)
{
if (topo == NULL)
  printf("\nEstoque vazio.\n");
else
 {
  printf("\nDigite o dia: ");
  scanf("%d", &dia);
  printf("\nDigite o mês: ");
  scanf("%d", &mes);
  printf("\nDigite o ano (com 4 dígitos): ");
  scanf("%d", &ano);
  estoque=0;
  aux=topo;
  while (aux!=NULL)
            {
            if (aux->dia>=dia && aux->mes>=mes && aux->ano>=ano)
            {
             if (aux->cod_op==1)
             printf("\nCompra em %d/%d/%d", aux->dia, aux->mes, aux->ano);
                        if (aux->cod_op==2)
                        printf("\nVenda em %d/%d/%d", aux->dia, aux->mes, aux-
```

```
>ano);
                        if (aux->cod_op==3)
                printf("\nPerda em %d/%d/%d", aux->dia, aux->mes, aux->ano);
                if (aux->cod_op==4)
                printf("\nDevolução em %d/%d/%d", aux->dia, aux->mes, aux->ano);
                printf(" Produto de código = &d  %d", aux->cod_prod, aux ->qtde);
                        }
                   aux=aux->posi;
                }
          }
         }
         if (op == 4)
         {
          if (topo == NULL)
           printf("\nEstoque vazio.\n");
           else
           {
                estoque = 0;
                aux=topo;
                while (aux!=NULL)
                {
                        if (aux->cod_op==1 || aux->cod_op==4)
                        estoque=estoque+aux->qtde;
                        else
                        estoque=estoque-aux->qtde;
                 aux=aux->posi;
```

```
            }
            printf("\nQuantidade em estoque: %d", estoque);
   }
}
if (op == 5) //consultar total de perdas
{
  if (topo == NULL)
  printf("\nEstoque vazio.\n");
  else
  {
            estoque = 0;
            aux=topo;
            while (aux!=NULL)
            {
                        if (aux->cod_op==3)
                        estoque=estoque+aux->qtde;
            aux=aux->posi;
            }
            printf("\nTotal de perdas = %d", estoque);
  }
}
if (op == 6)
{
   if (topo == NULL)
   printf("\nEstoque vazio.\n");
   else
```

```
        {
          aux = topo;
          topo = topo->posi;
          delete(aux);
          printf("\nÚltima operação excluída.\n");
        }
      }
      if (op == 7)
      {
         if (topo == NULL)
           printf("\nEstoque vazio.\n");
         else
         {
             aux = topo;
             while (aux != NULL)
             {
                topo = aux->posi;
                delete(aux);
                aux = topo;
             }
             printf("\nPilha esvaziada.\n");
         }
      }
      system("PAUSE");
   }
   while (op != 8);
}
```

ACOMPANHE NA WEB

FEOFILOFF, Paulo. Pilhas.

A estrutura de dados pilha é uma das estruturas de dados que suportam a inserção e remoção de elementos. Ela respeita a regra que o primeiro elemento inserido na pilha é o último a ser removido. Tal regra é conhecida pela sigla LIFO (Last-In, First-Out).

Disponível em: <http://www.ime.usp.br/~pf/algoritmos/aulas/pilha.html>. Acesso em: 21 maio 2014.

ROCHA, Fabio Gomes. Pilha: Fundamentos e implementação da estrutura em Java.

Uma questão importante no estudo da computação é o entendimento das estruturas de dados, entre as quais temos filas, pilhas, listas ligadas, entre outras. Entenda aqui o funcionamento das pilhas e como implementar uma pilha simples.

Disponível em: <http://www.devmedia.com.br/pilhas-fundamentos-e-implementacao-da-estrutura-em-java/28241>. Acesso em: ago. 2014.

Estrutura de Dados I - Aula3 Parte1 Pilha.

Você compreenderá o conceito da estrutura de dados pilha e vai explorar as diferenças entre as variações de implementação da pilha.

Disponível em: <https://www.youtube.com/watch?v=yb9XdY57VTk>. Acesso em: ago. 2014.

Tempo: 10:08.

Pilhas - Vídeo Aula [Java].

Neste vídeo, você compreenderá o conceito de pilha como uma estrutura de dados, identificará as diferenças entre a implementação de pilha, utilizando alocação de memória encadeada.

Disponível em: <https://www.youtube.com/watch?v=hFVdFIs1Do4>. Acesso em: ago. 2014.

Tempo: 27:51.



AGORA É A SUA VEZ

Instruções:

Agora, chegou a sua vez de exercitar seu aprendizado. A seguir, você encontrará algumas questões de múltipla escolha e dissertativas. Leia cuidadosamente os enunciados e atente-se para o que está sendo pedido.

Questão 1

Com o intuito de verificar as habilidades de programação adquiridas até agora, a atividade proposta é a solução de um problema simples utilizando os recursos de programação estudados. Tais conhecimentos são extremamente necessários na sequência dos conteúdos, pois haverá muitos exemplos e exercícios baseados na linguagem C.

Problema proposto: Implementar um programa em C que carregue um vetor com 10 números inteiros positivos. O programa deverá executar as operações a seguir:

1) Inserir número.

2) Listar todos os números.

3) Consultar um número.

4) Excluir um número.

Observação: O vetor não precisa ser ordenado e deverá permitir a inserção de números repetidos.

Questão 2

Observe o código, a seguir, e classifique o tipo de pilha utilizada com relação à estrutura de programação:

```
aux=new(pilha);

scanf("%d", &aux->num);

aux->prox=topo;

topo=aux;
```

a) Dinâmica.

b) Estática.

c) Incremental.

d) Analítica.

e) Nenhuma das alternativas anteriores.

Questão 3

A estrutura de dados do tipo pilha com relação às operações de inserção e remoção pode ser classificada como:

a) FIFO.

b) LIFO.

c) LOFO.

d) FOFI.

e) Nenhuma das alternativas anteriores.

Questão 4

Com base na estrutura de programação da linguagem C, criar um programa que carregue uma pilha dinâmica para manipular uma estrutura contendo a nota1, a nota2 e a média de 5 alunos. Resolver os itens a seguir:

a) Ler as notas dos 5 alunos.

b) Calcular as médias aritméticas das notas.

c) Listar as médias de todos os alunos.

d) Ao finalizar o programa, esvaziar a pilha.

Questão 5

Com base na estrutura de programação da linguagem C, criar um programa que carregue uma pilha dinâmica heterogênea para resolver os itens a seguir:

1) Cadastrar funcionário.

2) Consultar todos os funcionários.

3) Consultar a média salarial.

4) Conceder aumento porcentual.

5) Excluir um funcionário.

6) Sair.

Observação: A estrutura desta pilha armazenará os seguintes dados:

- Nome do funcionário.
- Salário.



FINALIZANDO

Os conceitos e exemplos práticos em C apresentados neste tema deixaram bem claro que a estrutura de dados do tipo pilha é muito versátil na solução de problemas do cotidiano. Sua estrutura permite armazenar tipos de dados variados, possibilitando o trabalho com estruturas homogêneas e heterogêneas.

As operações básicas de inserção, consulta, remoção e esvaziamento da pilha foram bem-exploradas nos exemplos e exercícios propostos, bem como as estruturas de pilha estática, dinâmica, homogênea e heterogênea com exemplos em C para cada uma das estruturas.

REFERÊNCIAS

FEOFILOFF, Paulo. Pilhas. Disponível em: <http://www.ime.usp.br/~pf/algoritmos/aulas/pilha.html>. Acesso em: 21 maio 2014.

ROCHA, Fabio Gomes. Pilhas: Fundamentos e implementação da estrutura em Java. Disponível em: <http://www.devmedia.com.br/pilhas-fundamentos-e-implementacao-da-estrutura-em-java/28241>. Acesso em: 21 maio 2014.

ROSSINI, Alexandre et al. Estrutura de Dados I - Aula3 Parte1 Pilha. Disponível em: <https://www.youtube.com/watch?v=yb9XdY57VTk>. Acesso em: 09/06/2014.

TENENBAUM, Aaron M.; LANGSAM, Yedidyah; AUGENSTEIN, Moshe J. Estruturas de dados usando C. São Paulo: Makron Books, 1995.

GLOSSÁRIO

Estrutura de Dados: na Computação, estrutura de dados é uma forma específica de organização e armazenamento de dados para que sejam utilizados de forma eficaz. As estruturas de dados e seus algoritmos são muito utilizados na Ciência da Computação em diversas áreas do conhecimento e com as mais diferentes finalidades na solução de problemas computacionais.

Heterogênea: aquilo que é composto por elementos diferentes, desiguais.

Homogênea: cujas partes são da mesma natureza ou estão estreitamente ligadas. Idêntica, igual, análoga.

Filas



POR DENTRO DO TEMA

A fila é uma **estrutura de dados** capaz de representar conjuntos de dados organizados em ordem linear. Quando se utilizam vetores nas representações dessas estruturas, são usados endereços contíguos de memória do computador e a ordem linear, que são indicados pelos índices dos vetores, considerando-se que sua implementação, em algumas situações, pode exigir maior esforço computacional. Tais representações denominam-se filas estáticas. Entretanto, quando as estruturas de dados do tipo fila são representadas por estruturas que contêm o dado e também um ponteiro para o próximo elemento, têm-se elementos encadeados, sendo, portanto, denominadas filas dinâmicas.

Quando um elemento de uma fila contém apenas um tipo de dado, como um número, denomina-se fila **homogênea**, e quando um elemento de uma fila contém um dado composto, como o nome e o salário de um funcionário, denomina-se fila **heterogênea**.

A estrutura denominada fila é considerada do tipo FIFO (First In First Out), ou seja, o primeiro elemento inserido será o primeiro a ser removido. Nessa estrutura, cada elemento armazena um ou vários dados (estrutura homogênea ou heterogênea, respectivamente) e um ponteiro para o próximo elemento, permitindo o encadeamento e mantendo a estrutura linear. Nesse tipo de estrutura, serão abordadas as seguintes operações: inserir na fila, consultar toda a fila, remover e esvaziá-la. A estrutura do tipo fila possui um ponteiro denominado INICIO, em que as remoções acontecem, e um denominado FIM, em que as inserções acontecem. Assim, as operações ocorrem nas duas extremidades da estrutura.

A operação de inserção na fila sempre realiza operações básicas, como a de atribuição, para atualizar o INICIO e FIM da fila. O mesmo ocorre no caso da remoção para atualizar o INICIO da fila. Logo, são operações de tempo constante e gastam tempo O(1).

Já a operação de consultar toda a fila percorre todos os elementos armazenados nela. Considerando que uma fila contém n elementos, o tempo de execução será O(n).

A operação de esvaziamento da fila consiste em remover todos os seus elementos. O tempo gasto nessa operação depende da linguagem de programação que está sendo utilizada. Na linguagem C, é necessário deslocar cada um dos elementos da fila, gastando tempo proporcional ao tamanho dela, ou seja, O(n).

Exemplos em C

Fila Estática e Homogênea

São estruturas com tamanho pré-definido, que não podem ser alteradas durante a execução do programa, manipulando só um tipo de dado. No caso do exemplo, a variável é do tipo inteiro fila.

```
#include<iostream>
#include<stdio.h>
int main()
{
  int fila[5];
  int op,i,j;
  i=0;
  do
  {
        system("CLS");
        printf("\n\t.MENU.");
        printf("\n1. Inserir");
        printf("\n2. Consultar inicio");
        printf("\n3. Consultar fim");
        printf("\n4. Consultar toda fila");
        printf("\n5. Excluir");
        printf("\n6. Esvaziar");
        printf("\n7. Sair");
        printf("\nDigite sua opção: ");
```

```
scanf("%d", &op);
switch(op)
{
  case(1):
  {
          if(i==5)
            printf("\nFila cheia.\n");
          else
          {
            printf("\nDigite o número a inserir: ");
            scanf("%d", &fila[i]);
            i++;
            printf("\nNúmero inserido.\n");
            break;
          }
  }
  case(2):
  {
          if(i==0)
            printf("\nFila vazia.\n");
          else
            printf("\nInicio da fila: %d", fila[0]);
          break;
  }
  case(3):
  {
```

```
        if(i==0)
          printf("\nFila vazia.\n");
        else
          printf("\nFim da fila: %d", fila[i-1]);
        break;
}
case(4):
{
        if(i==0)
          printf("\nFila vazia.\n");
        else
        {
          printf("\nToda a fila: ");
          for(j=0;j<i;j++)
                printf("%d ", fila[j]);
        }
        break;
}
case(5):
{
        if(i==0)
          printf("\nFila vazia.\n");
        else
        {
          for(j=0;j<i-1;j++)
                fila[j]=fila[j+1];
```

```
                printf("\nNúmero excluído.\n");
                i--;
              }
              break;
        }
        case(6):
        {
              if(i==0)
                printf("\nFila vazia.\n");
              else
              {
                i=0;
                printf("\nFila esvaziada");
              }
              break;
        }
        default:
        {
              if(op!=7)
                printf("\nOpção inválida.\n");
        }
      }
      system("PAUSE");
  }
  while(op!=7);
}
```

Fila Estática e Heterogênea

São estruturas com tamanho pré-definido, que não podem sofrer alteração durante a execução do programa, manipulando mais de um tipo de dado. No caso do exemplo, as variáveis são do tipo inteiro codigo e do tipo real nota.

```
#include<iostream>
#include<stdio.h>
int main()
{
  struct
  {
        int codigo;
        float nota;
  }fila[5];
  int op,i,j;
  i=0;
  do
  {
        system("CLS");
        printf("\n\t.MENU.");
        printf("\n1. Inserir");
        printf("\n2. Consultar inicio");
        printf("\n3. Consultar fim");
        printf("\n4. Consultar toda fila");
        printf("\n5. Excluir");
        printf("\n6. Esvaziar");
        printf("\n7. Sair");
        printf("\nDigite sua opção: ");
```

```
scanf("%d", &op);
switch(op)
{
  case(1):
  {
          if(i==5)
            printf("\nFila cheia.\n");
          else
          {
            printf("\nDigite o código do aluno: ");
            scanf("%d", &fila[i].codigo);
            printf("\nDigite a nota do aluno: ");
            scanf("%f", &fila[i].nota);
            i++;
            printf("\nDados inseridos.\n");
            break;
          }
  }
  case(2):
  {
          if(i==0)
            printf("\nFila vazia.\n");
          else
          {
            printf("\nInicio da fila: \n");
            printf("\nCódigo: %d", fila[0].codigo);
```

```
            printf("\nNota: %2.2f\n", fila[0].nota);
          }
          break;
}
case(3):
{
          if(i==0)
            printf("\nFila vazia.\n");
          else
          {
            printf("\nFim da fila: \n");
            printf("\nCódigo: %d", fila[i-1].codigo);
            printf("\nNota: %2.2f\n", fila[i-1].nota);
          }
          break;
}
case(4):
{
          if(i==0)
            printf("\nFila vazia.\n");
          else
          {
            printf("\nToda a fila: \n");
            for(j=0;j<i;j++)
            {
                        printf("\nCódigo: %d", fila[j].codigo);
```

```
                printf("\nNota: %2.2f\n", fila[j].nota);
            }
        }
        break;
}
case(5):
{
        if(i==0)
          printf("\nFila vazia.\n");
        else
        {
          for(j=0;j<i-1;j++)
          {
                fila[j].codigo=fila[j+1].codigo;
                fila[j].nota=fila[j+1].nota;
          }
          printf("\nDados excluidos.\n");
          i--;
        }
        break;
}
case(6):
{
        if(i==0)
          printf("\nFila vazia.\n");
        else
```

```
                {
                  i=0;
                  printf("\nFila esvaziada.\n");
                }
                break;
          }
          default:
          {
                if(op!=7)
                  printf("\nOpção inválida.\n");
          }
        }
        system("PAUSE");
    }
    while(op!=7);
}
```

Fila Dinâmica e Homogênea

São estruturas que utilizam ponteiros para indexar endereços de variáveis, permitindo a inserção dinâmica de dados durante a execução do programa e manipulando apenas um tipo de dado. No caso do exemplo, a variável é do tipo inteiro fila.

```
#include<iostream>
#include<stdio.h>
int main()
{
  struct fila
  {
```

```
        int num;
        fila *prox;
};
fila *inicio,*fim,*aux;
int op;
inicio=NULL;
do
{
        system("CLS");
        printf("\n\tMENU");
        printf("\n1. Inserir");
        printf("\n2. Consultar inicio");
        printf("\n3. Consultar fim");
        printf("\n4. Consultar toda fila");
        printf("\n5. Excluir");
        printf("\n6. Esvaziar");
        printf("\n7. Sair");
        printf("\nDigite sua opção: ");
        scanf("%d", &op);
        switch(op)
        {
                case(1):
                {
                  aux=new(fila);
                  printf("\nDigite o número a inserir: ");
                  scanf("%d", &aux->num);
```

```
  if(inicio==NULL)
            inicio=aux;
  else
            fim->prox=aux;
  fim=aux;
  fim->prox=NULL;
  printf("\nNúmero inserido.\n");
  break;
}
case(2):
{
  if(inicio==NULL)
            printf("\nFila vazia.\n");
  else
            printf("\nNúmero inicial: %d\n", inicio->num);
  break;
}
case(3):
{
  if(inicio==NULL)
            printf("\nFila vaizia.\n");
  else
            printf("\nNúmero final: %d\n", fim->num);
  break;
}
case(4):
```

```
{
  if(inicio==NULL)
            printf("\nFila vazia.\n");
  else
  {
            printf("\nToda a fila: ");
            aux=inicio;
            while(aux!=NULL)
            {
                        printf("%d ", aux->num);
                        aux=aux->prox;
            }
  }
  break;
}
case(5):
{
  if(inicio==NULL)
            printf("\nFila vazia.\n");
  else
  {
            aux=inicio;
            inicio=inicio->prox;
            delete(aux);
            printf("\nNúmero excluido.\n");
  }
```

```
            break;
        }
        case(6):
        {
            if(inicio==NULL)
                printf("\nFila vazia.\n");
            else
            {
                aux=inicio;
                while(aux!=NULL)
                {
                    inicio=inicio->prox;
                    delete(aux);
                    aux=inicio;
                }
                printf("\nFila esvaziada.\n");
            }
            break;
        }
        default:
        {
            if(op!=7)
                printf("\nOpção invalida.\n");
        }
}
system("PAUSE");
```

```
  }
  while(op!=7);
}
```

Fila Dinâmica e Heterogênea

São estruturas que utilizam ponteiros para indexar endereços de variáveis permitindo a inserção dinâmica de dados durante a execução do programa e manipulando mais de um tipo de dado. No caso do exemplo, as variáveis são do tipo inteiro codigo e do tipo real salario.

```
#include<iostream>
#include<stdio.h>
int main()
{
  struct fila
  {
        int codigo;
        float salario;
        fila *prox;
  };
  fila *inicio,*fim,*aux;
  int op;
  inicio=NULL;
  do
  {
        system("CLS");
        printf("\n\tMENU");
        printf("\n1. Inserir dados");
        printf("\n2. Consultar dados do inicio");
```

```
printf("\n3. Consultar dados do fim");
printf("\n4. Consultar toda fila");
printf("\n5. Excluir dados");
printf("\n6. Esvaziar fila");
printf("\n7. Sair");
printf("\nDigite sua opŧao: ");
scanf("%d", &op);
switch(op)
{
        case(1):
        {
          aux=new(fila);
          printf("\nDigite o código do funcionario: ");
          scanf("%d", &aux->codigo);
          printf("\nDigite o salario: ");
          scanf("%f", &aux->salario);
          if(inicio==NULL)
                  inicio=aux;
          else
                  fim->prox=aux;
          fim=aux;
          fim->prox=NULL;
          printf("\nDados inseridos.\n");
          break;
        }
        case(2):
```

```
{
  if(inicio==NULL)
            printf("\nFila vazia.\n");
  else
  {
            printf("\nDados iniciais:\n");
            printf("\nCódigo: %d", inicio->codigo);
            printf("\nSalario: %4.2f\n", inicio->salario);
  }
  break;
}
case(3):
{
  if(inicio==NULL)
            printf("\nFila vazia.\n");
  else
  {
            printf("\nDados finais:\n");

            printf("\nCódigo: %d", fim->codigo);
            printf("\nSalario: %4.2f\n", fim->salario);
  }
  break;
}
case(4):
{
```

```
  if(inicio==NULL)
            printf("\nFila vazia.\n");
  else
  {
            printf("\nToda a fila:\n");
            aux=inicio;
            while(aux!=NULL)
            {
            printf("\nCódigo: %d", aux->codigo);
            printf("\nSalario: %4.2f\n", aux->salario);

                        aux=aux->prox;
            }
  }
  break;
}
case(5):
{
  if(inicio==NULL)
            printf("\nFila vazia.\n");
  else
  {
            aux=inicio;
            inicio=inicio->prox;
            delete(aux);
            printf("\nDados excluidos.\n");
```

```
        }
        break;
      }
      case(6):
      {
        if(inicio==NULL)
              printf("\nFila vazia.\n");
        else
        {
              aux=inicio;
              while(aux!=NULL)
              {
                    inicio=inicio->prox;
                    delete(aux);
                    aux=inicio;
              }
              printf("\nFila esvaziada.\n");
        }
        break;
      }
      default:
      {
        if(op!=7)
              printf("\nOpçâo inválida.\n");
      }
}
```

```
        system("PAUSE");
  }
  while(op!=7);
}
```

Exemplo de Fila e Pilha Dinâmica Heterogênea no Mesmo Programa

```
#include <iostream>
#include <stdio.h>
int main()
{
        //Criando a estrutura da fila de dependentes
        struct dependente
        {
         char nome_dep[40];
         dependente *prox_dep;
        };

        //Criando a estrutura da pilha de funcionários
        struct funcionario
        {
         int num;
         char nome_func[40];
         float valor, qtde;
         dependente *dep;
         funcionario *prox_func;
        };
```

```
funcionario *topo_func, *aux_func;
dependente *aux_dep, *fim_dep;
topo_func = NULL;
int op,numero,nf,achou,soma;
float sal_bruto, salario, total;
   numero = 0;   // variável utilizada para gerar o número do funcionário

do
{
    system("CLS");
    printf("Menu");
    printf("\n1 - Cadastrar funcionário");
    printf("\n2 - Cadastrar dependente");
    printf("\n3 - Consultar salário");
    printf("\n4 - Consultar folha de pagamento");
    printf("\n5 - Excluir dependente");
    printf("\n6 - Excluir funcionário");
    printf("\n7 - Sair");
    printf("\nDigite sua opção: ");
    scanf("%d", &op);
    if (op <1 || op > 7)
    printf("\nOpção inválida.\n");
    if (op == 1)
    {
            aux_func = new(funcionario);
            numero = numero + 1;
```

```
                aux_func -> num = numero;
                printf("\nDigite o nome do funcionário ");
                scanf("%s",&aux_func->nome_func);
                printf("\nDigite a quantidade de horas trabalhadas ");
                scanf("%f", &aux_func->qtde);
                printf("\nDigite o valor da hora trabalhada ");
                scanf("%f", &aux_func->valor);
                aux_func->dep = NULL;
                aux_func->prox_func = topo_func;
                topo_func = aux_func;
                printf("\nFuncionário cadastrado.\n");
        }
        if (op == 2)
        {
                if (topo_func == NULL)
                 printf("\nPilha de funcionários vazia.\n");
                else
                {
                printf("\nDigite o número do funcionário que receber o
dependente: ");
                scanf("%d", &nf);
                achou = 0;
                aux_func = topo_func;
                while (aux_func !=NULL && achou == 0)
                {
                        if (aux_func->num == nf)
```

```
                    achou = 1;
            else
                    aux_func = aux_func -> prox_func;
    }
    if (achou == 0)
     printf("\nNão existe funcionário cadastrado com o número %d\n",
nf);
    else
    {
            aux_dep = new(dependente);
            printf("\nDigite o nome do dependente: ");
            scanf("%s",&aux_dep->nome_dep);
            if (aux_func->dep == NULL)
                    aux_func->dep = aux_dep;
            else
                    {
                     fim_dep = aux_func->dep;
                     while (fim_dep->prox_dep !=NULL)
                     {
                             fim_dep = fim_dep->prox_dep;
                     }
                     fim_dep->prox_dep = aux_dep;
                    }
             aux_dep->prox_dep = NULL;
            printf("\nDepentende cadastrado com sucesso.\n");
     }
```

```
            }
        }
        if (op == 3)
        {
            if (topo_func == NULL)
             printf("\nPilha de funcionários vazia.\n");
            else
            {
            printf("\nDigite o número do funcionário para consultar o salário:
");
            scanf("%d", &nf);
            achou = 0;
            aux_func = topo_func;
            while (aux_func !=NULL && achou == 0)
            {
                if (aux_func->num == nf)
                    achou = 1;
                else
                    aux_func = aux_func -> prox_func;
            }
            if (achou == 0)
             printf("\nNão existe funcionário cadastrado com o número %d\n", nf);
            else
             {
                sal_bruto = aux_func -> qtde * aux_func -> valor;
                soma = 0;
```

```
                fim_dep = aux_func->dep;
                 while (fim_dep !=NULL)
                        {
                                soma = soma + 1;
                                fim_dep = fim_dep->prox_dep;
                        }
                salario = sal_bruto + (soma * 50);
                printf("\nSalário do funcionário = %4.2f\n", salario);
         }
        }
}
if (op == 4)
{
        if (topo_func == NULL)
         printf("\nPilha de funcionários vazia.\n");
        else
        {
        total = 0;
        aux_func = topo_func;
        while (aux_func !=NULL)
        {
                sal_bruto = aux_func -> qtde * aux_func -> valor;
                soma = 0;
                fim_dep = aux_func->dep;
                 while (fim_dep !=NULL)
                        {
```

```
                                        soma = soma + 1;
                                        fim_dep = fim_dep->prox_dep;
                                }
                        salario = sal_bruto + (soma * 50);
                        total = total + salario;
                        aux_func = aux_func->prox_func;
                }
                printf("\nTotal da folha de pagamento = %4.2f\n", total);
                }
        }
        if (op == 5)
        {
                if (topo_func == NULL)
                  printf("\nPilha de funcionários vazia.\n");
                else
                {
                printf("\nDigite o número do funcionário do qual o dependente ser
excluído: ");
                scanf("%d", &nf);
                achou = 0;
                aux_func = topo_func;
                while (aux_func !=NULL && achou == 0)
                {
                        if (aux_func->num == nf)
                                achou = 1;
                        else
```

```
                                aux_func = aux_func -> prox_func;
                }
                if (achou == 0)
                 printf("\nNão existe funcionário cadastrado com o número %d\n",
nf);
                else
                 {
                        if (aux_func ->dep == NULL)
                                 printf("\nEste funcionário não tem dependentes
para exclusão.\n");
                        else
                                {
                                 aux_dep = aux_func ->dep;
                                 aux_func -> dep = aux_dep->prox_dep;
                                 delete(aux_dep);
                                 printf("\nDependente excluído com sucesso.\n");
                                }
                 }
              }
          }
          if (op == 6)
          {
                if (topo_func == NULL)
                 printf("\nPilha de funcionários vazia.\n");
                else
                {
                aux_func = topo_func;
```

```
                aux_dep = aux_func ->dep;
                while (aux_dep != NULL)
                {
                        aux_func -> dep = aux_dep->prox_dep;
                        delete(aux_dep);
                        aux_dep = aux_func -> dep;
                }
                topo_func = topo_func->prox_func;
                delete(aux_func);
                printf("\nFuncionário e seus dependentes excluídos com
sucesso.\n");
                }
          }
          system("PAUSE");
         }while (op !=7);
    }
```



ACOMPANHE NA WEB

Aula sobre Fila Estática e Dinâmica em JAVA (texto)

O que diferencia a fila da pilha é a ordem de saída dos elementos: enquanto na pilha "o último que entra é o primeiro que sai", na fila "o primeiro que entra é o primeiro que sai" (a sigla FIFO – first in, first out – é usada para descrever essa estratégia).

Disponível em: <http://www.din.uem.br/~ronaldo/LivroAED-Capitulos-7-Estruturas.pdf>. Acesso em: 16 ago. 2014.

Estruturas de Dados em Java Pilha e Fila (apresentação)

A ideia fundamental da fila é que só podemos inserir um novo elemento no final da fila e só podemos retirar o elemento do início. Como dito, o que diferencia a fila da pilha é a ordem de saída dos elementos: enquanto na pilha "o último que entra é o primeiro que sai", na fila "o primeiro que entra é o primeiro que sai".

Disponível em: <http://www.ime.usp.br/~pf/estruturas-de-dados/aulas/stack.html>. Acesso em: 16 ago. 2014.

Estrutura de dados – Fila

Neste vídeo, você pode ver conceitos básicos do tipo abstrato fila, além de operações que descrevem o seu comportamento.

Disponível em: <https://www.youtube.com/watch?v=ju8jdYwd8lo>. Acesso em: 16 ago. 2014

Tempo: 09:43

Estrutura de dados I

Este vídeo ajuda a compreender o conceito de fila como uma estrutura de dados, além de identificar as diferenças em fila com utilização de alocação de memória sequencial e encadeada.

Disponível em: <https://www.youtube.com/watch?v=6AwcBkYoj3c>. Acesso em: 16 ago. 2014.

Tempo: 10:03



AGORA É A SUA VEZ

Instruções:

Agora, chegou a sua vez de exercitar seu aprendizado. A seguir, você encontrará algumas questões de múltipla escolha e dissertativas. Leia cuidadosamente os enunciados e atente-se para o que está sendo pedido.

Questão 1

Com o intuito de verificar as habilidades de programação adquiridas até agora, propõe-se aqui uma atividade que corresponde a uma solução de um problema

simples utilizando os recursos de programação estudados no tema anterior. Tais conhecimentos são extremamente necessários na sequência dos conteúdos, pois haver muitos exemplos e exercícios baseados na linguagem C.

Problema proposto: implementar um programa em C que carregue uma pilha dinâmica homogênea com números inteiros positivos. O programa deverá executar as operações a seguir:

I. Inserir número na pilha.

II. Listar pilha.

III. Excluir um número da pilha.

Questão 2

Observe o código seguinte e classifique o tipo de fila utilizada com relação à estrutura de programação:

```
struct

{

int codigo;

      float sal;

} fila[5];

.

.

.

std::cin>>fila[topo].codigo;

std::cin>>fila[topo].sal;
```

a) Incremental.

b) Dinâmica.

c) Estática.

d) Analítica.

e) Nenhuma das alternativas é verdadeira.

Questão 3

Observe o código a seguir, classificando o tipo de fila utilizada com relação à estrutura de programação, e depois escolha a alternativa correta.

```
struct
{
int codigo;
      float sal;
} fila[5];
.
.
.
std::cin>>fila[topo].codigo;
std::cin>>fila[topo].sal;
```

a) Homogênea.

b) Dinâmica.

c) Analítica.

d) Heterogênea.

e) Nenhuma das alternativas está correta.

Questão 4

Com base na estrutura de programação da linguagem C, crie um programa que carregue uma fila dinâmica para manipular uma estrutura contendo o nome e salário de 6 funcionários, resolvendo os itens seguintes:

- Cadastrar 6 funcionários na fila.
- Mostrar o imposto e o salário de todos os funcionários da fila.
- Mostrar os funcionários cujo nome comece com uma letra digitada.
- Esvaziar a fila antes de sair do programa.

Questão 5

Com base na estrutura de programação da linguagem C, crie um programa que carregue uma fila dinâmica para manipular uma estrutura contendo o nome e a média de 10 alunos. Resolva os itens a seguir:

- Cadastrar aluno.
- Mostrar o nome e a média dos aprovados.
- Mostrar o nome e a média dos reprovados.
- Esvaziar a fila antes de sair do programa.

Obs.: é considerado aprovado o aluno com média >= 6.



FINALIZANDO

Os conceitos e exemplos práticos em C apresentados neste tema deixaram bem claro que a estrutura de dados do tipo fila é muito versátil na solução de problema do cotidiano. Sua estrutura permite armazenar tipos de dados variados, possibilitando o trabalho com estruturas homogêneas e heterogêneas.

As operações básicas de inserção, consulta, remoção e esvaziamento da fila foram bem exploradas nos exemplos e exercícios propostos, bem como as estruturas de fila estática, dinâmica, homogênea e heterogênea com exemplos em C para cada uma das estruturas.

REFERÊNCIAS

AZEVEDO, Gilbert. Estruturas de Dados em Java. Pilha e Fila. Disponível em: <http://www.ime.usp.br/~pf/estruturas-de-dados/aulas/stack.html>. Acesso em: 21 maio 2014.

COSTA, Sérgio Souza. Estrutura de Dados – Filas. Vídeo. Disponível em: <https://www.youtube.com/watch?v=ju8jdYwd8lo>. Acesso em: 15 jun. 2014.

HEIL JUNIOR, Manfred. Aula 06 - Fila estática e dinâmica em JAVA. Disponível em: <http://www.din.uem.br/~ronaldo/LivroAED-Capitulos-7-Estruturas.pdf>. Acesso em: 21 maio 2014.

ROSSINI, Alexandre; TOLETINO, Carlos H.; COELHO, Alex; ANTONIO, Marcos; GONÇALVES, Yzaac. Estrutura de Dados I - Aula4 Parte1 Fila. Disponível em: <https://www.youtube.com/watch?v=6AwcBkYoj3c>. Acesso em: 15 jun. 2014.

TENENBAUM, Aaron M.; LANGSAM, Yedidyah; AUGENSTEIN, Moshe J. Estruturas de dados usando C. São Paulo: Makron Books, 1995.

GLOSSÁRIO

Estrutura de dados: na computação, estrutura de dados é uma forma específica de organização e armazenamento de dados para que estes sejam utilizados de forma eficaz. As estruturas de dados e seus algoritmos são muito utilizados na Ciência da Computação, em diversas áreas do conhecimento, e com as mais diferentes finalidades na solução de problemas computacionais.

Homogênea: cujas partes são da mesma natureza ou estão estreitamente ligadas. Idêntica, igual, análoga.

Heterogênea: aquilo que é composto por elementos diferentes, desiguais.

Lista



POR DENTRO DO TEMA

O trabalho com conjuntos de dados que podem representar coleções de números, dados de um cliente, dados de um fornecedor, entre outros, é muito difundido e importante para a Computação.

Esses conjuntos podem ser chamados de conjuntos dinâmicos, pois os algoritmos que os manipulam fazem com que eles aumentem, diminuam ou sofram alterações durante sua manipulação. Algumas operações sobre os conjuntos dinâmicos podem ser: inserir elemento, excluir elemento, buscar elemento, encontrar o maior, o menor, somar elementos, alterar elementos, entre outras.

A **estrutura de dados** do tipo lista representa um conjunto de dados organizados em ordem linear. Quando a estrutura lista é representada por um arranjo, ou seja, quando se utilizam vetores na representação, tem-se o uso de endereços contíguos de memória do computador, e a ordem linear é determinada pelos índices do vetor, o que exige um maior esforço computacional em algumas situações. Tal representação denomina-se lista estática.

Quando a estrutura lista é representada por elementos que, além de conter o dado, possuem também um ponteiro para o próximo elemento, ou seja, elementos encadeados, tem-se a representação denominada lista dinâmica.

Tipos de Lista e Exemplos em C

Lista Simplesmente Encadeada e Não Ordenada

Na estrutura do tipo lista simplesmente **encadeada** e não ordenada, cada elemento pode armazenar um ou mais dados (estrutura homogênea ou **heterogênea**, respectivamente) e um ponteiro para o próximo elemento, que permite o encadeamento e mantém a estrutura linear. A seguir, apresenta-se um exemplo com as operações: inserir no início da lista, inserir no fim, consultar toda a lista, consultar um número na lista, remover um elemento qualquer e esvaziá-la.

Exemplo:

```
#include <iostream>
#include <stdio.h>

int main()
{
        struct lista
        {
                int num;
                lista *prox;

        };

        lista *inicio,*fim,*aux,*aux2;
        inicio=NULL;

        int op,qtde,nc,achou;
    do
    {
      system("CLS");
      printf("\nMENU");
      printf("\n1 - Inserir no inicio");
      printf("\n2 - Inserir no fim");
      printf("\n3 - Consultar toda a lista");
      printf("\n4 - Consultar um nº da lista");
      printf("\n5 - Excluir nº da lista");
```

```
printf("\n6 - Esvaziar a lista");
printf("\n7 - Sair");
printf("\nDigite a opcao: ");
scanf("%d", &op);

 if (op<1||op>7)
 printf("\nOpção inválida.\n");
 else
 {
        if (op==1)
        {
                aux=new(lista);
                printf("\nDigite um número: ");
                scanf("%d", &aux->num);
                if (inicio==NULL)
                fim=aux;

                aux->prox=inicio;
                inicio=aux;

                printf("\nNúmero inserido.\n");
        }
        if (op==2)
        {
                aux=new(lista);
                printf("\nDigite um número: ");
```

```
            scanf("%d", &aux->num);
            if (inicio==NULL)
            inicio=aux;
            else
            fim->prox=aux;
            fim=aux;
            fim->prox=NULL;
            printf("\nNúmero inserido.\n");
}
if (op==3)
{
            if (inicio==NULL)
            printf("\nLista vazia.\n");
            else
            {
                        aux=inicio;
                        while (aux!=NULL)
                        {
                                    printf("%d\n", aux->num);
                                    aux=aux->prox;
                        }
            }
}
if (op==4)
{
            if (inicio==NULL)
```

```
            printf("\nLista vazia.\n");
            else
            {
                  qtde=0;
                  printf("\nDigite o número a ser consultado: ");
                  scanf("%d", &nc);
                  aux=inicio;
                  while (aux!=NULL)
                  {
                        if (aux->num==nc)
                        qtde++;

                        aux=aux->prox;
                  }
                  if (qtde==0)
                  printf("\nNúmero não encontrado.\n");
                  else if (qtde==1)
                  printf("\nNúmero %d aparece 1 vez.\n", nc);
                  else
                  printf("\nNúmero %d aparece %d vezes.\n", nc, qtde);
            }
}
if (op==5)
{
            if (inicio==NULL)
            printf("\nLista vazia.\n");
```

```
else
{
printf("\nDigite o número que deseja excluir: ");
scanf("%d", &nc);
aux=inicio;
achou=0;
while (aux!=NULL)
{
        if (aux->num==nc)
        {
        achou++;

                if (aux==inicio)
                {
                inicio=aux->prox;
                delete(aux);
                aux=inicio;
                }

                else if (aux==fim)
                {
                aux2=inicio;
                while (aux2->prox!=aux)
                aux2=aux2->prox;
                delete(aux);
                fim=aux2;
```

```
                    fim->prox=NULL;
                    aux=NULL;
                    }

                    else
                    {
                    aux2=inicio;
                    while (aux2->prox!=aux)
                    aux2=aux2->prox;
                    aux2->prox=aux->prox;
                    delete(aux);
                    aux=aux2->prox;
                    }
                }
                else
                aux=aux->prox;
            }
            if (achou==0)
            printf("\nNnúmero não existe na lista.\n");
            else if (achou==1)
            printf("\nFoi excluída %d ocorrência do nnúmero %d\n", achou, nc);
            else
            printf("\nForam excluídas %d ocorrências do número %d\n", achou, nc);
            }
}
if (op==6)
```

```
            {
                if (inicio==NULL)
                printf("\nLista vazia.\n");
                else
                {
                    aux=inicio;
                    while (aux!=NULL)
                    {
                        inicio=inicio->prox;
                        delete(aux);
                        aux=inicio;
                    }
                printf("\nLista esvaziada.\n");
                }
            }
        }
            system("PAUSE");
    }while (op!=7);
}
```

Lista Simplesmente Encadeada e Ordenada

Na estrutura do tipo lista simplesmente encadeada e ordenada, cada elemento pode armazenar um ou mais dados (estrutura homogênea ou heterogênea, respectivamente) e um ponteiro para o próximo elemento, que permite o encadeamento e mantém a estrutura linear. Tem-se, também, um campo denominado chave, por meio do qual determinada ordenação é mantida. Apresenta-se um exemplo com as seguintes operações: inserir na lista, consultar toda a lista,

consultar um número na lista, remover um elemento qualquer e esvaziá-la.

Exemplo:

```
#include <iostream>
#include <stdio.h>
int main()
{
      struct lista
      {
            int num;
            lista *prox;
      };

      lista *inicio,*fim,*aux,*aux2;
      inicio=NULL;
      int op,qtde,nc,achou;
   do
   {
     system("CLS");
     printf("\nMENU");
     printf("\n1 - Inserir na lista");
     printf("\n2 - Consultar toda a lista");
     printf("\n3 - Consultar um nº da lista");
     printf("\n4 - Excluir nº da lista");
     printf("\n5 - Esvaziar a lista");
     printf("\n6 - Sair");
     printf("\nDigite a opcao: ");
```

```
scanf("%d", &op);

 if (op<1||op>6)
 printf("\nOpcao invalida.\n");
 else
 {
        if (op==1)
        {
                aux=new(lista);
                printf("\nDigite o numero a ser inserido: ");
                scanf("%d", &aux->num);

                if (inicio==NULL)
                {
                fim=aux;
                aux->prox=inicio;
                inicio=aux;
                }
                else if (aux->num <= inicio->num)
                        {
                        aux->prox=inicio;
                        inicio=aux;
                        }

                        else if (aux->num >= fim->num)
                        {
```

```
                    fim->prox=aux;

                    aux->prox=NULL;
                    fim=aux;

                    }

                    else  //insercao no meio
                    {
                    aux2=inicio;
                    while (aux2->prox->num < aux->num)
                    aux2=aux2->prox;

                    aux->prox=aux2->prox;
                    aux2->prox=aux;
                    }
printf("\nNumero inserido.\n");
}

if (op==2)
{
        if (inicio==NULL)
        printf("\nLista vazia.\n");
        else
        {
                aux=inicio;
```

```
                while (aux!=NULL)
                {
                        printf("%d\n", aux->num);
                        aux=aux->prox;
                }
        }
}

if (op==3)
{
        if (inicio==NULL)
        printf("\nLista vazia.\n");
        else
        {
                qtde=0;
                printf("\nDigite o nº a ser consultado: ");
                scanf("%d", &nc);
                aux=inicio;
                while (aux!=NULL && aux->num<=nc)
                {
                        if (aux->num==nc)
                        qtde++;

                        aux=aux->prox;
                }
                if (qtde==0)
```

```
                    printf("\nNº nao encontrado.\n");
                    else if (qtde==1)
                    printf("\nNúmero %d aparece 1 vez.\n", nc);
                    else
                    printf("\nNúmero %d aparece %d vezes.\n", nc,
qtde);
            }
        }

        if (op==4)
        {
            if (inicio==NULL)
            printf("\nLista vazia.\n");
            else
            {
            printf("\nDigite o numero que deseja excluir: ");
            scanf("%d", &nc);
            aux=inicio;
            achou=0;
            while (aux!=NULL && aux->num<=nc)
            {
                if (aux->num==nc)
                {
                achou++;
                    if (aux==inicio)
                    {
```

```
                inicio=aux->prox;
                delete(aux);
                aux=inicio;
                }

                else if (aux==fim)
                {
                aux2=inicio;
                while (aux2->prox!=aux)
                aux2=aux2->prox;
                delete(aux);
                fim=aux2;
                fim->prox=NULL;
                aux=NULL;
                }

                else
                {
                aux2=inicio;
                while (aux2->prox!=aux)
                aux2=aux2->prox;
                aux2->prox=aux->prox;
                delete(aux);
                aux=aux2->prox;
                }
        }
```

```
                else
                aux=aux->prox;
        }
        if (achou==0)
        printf("\nNumero não encontrado.\n");
        else if (achou==1)
        printf("\nFoi excluída %d ocorrência do nnúmero %d\n", achou, nc);
        else
        printf("\nForam excluídas %d ocorrências do número %d\n", achou, nc);
        }
}


if (op==5)
{
        if (inicio==NULL)
        printf("\nLista vazia.\n");
        else
        {
                aux=inicio;
                while (aux!=NULL)
                {
                        inicio=inicio->prox;
                        delete(aux);
                        aux=inicio;
                }
```

```
                    printf("\nLista esvaziada.\n");
                    }
                }

        }
            system("PAUSE");
    }while (op!=6);
}
```

Lista Duplamente Encadeada e Não Ordenada

Na estrutura do tipo lista duplamente encadeada e ordenada, cada elemento pode armazenar um ou mais dados (estrutura homogênea ou heterogênea, respectivamente) e dois ponteiros. O primeiro para o próximo elemento, e o segundo para o elemento anterior. Esses ponteiros permitem o duplo encadeamento e mantêm a estrutura linear. Apresenta-se um exemplo com as seguintes operações: inserir no início da lista, inserir no fim, consultar toda a lista do início ao fim ou do fim ao início, consultar um número da lista, remover um elemento qualquer e esvaziá-la.

Exemplo:

```
#include <iostream>
#include <stdio.h>
int main()
{
    struct lista
    {
        int num;
        lista *ant,*prox;

    };
```

```
    lista *inicio,*fim,*aux,*aux2;
    inicio=NULL;
    int op,qtde,nc,achou;
do
{
  system("CLS");
  printf("\nMENU");
  printf("\n1 - Inserir no inicio");
  printf("\n2 - Inserir no fim");
  printf("\n3 - Consultar toda a lista do inicio para o fim");
  printf("\n4 - Consultar toda lista do fim para o inicio");
  printf("\n5 - Consultar um nº da lista");
  printf("\n6 - Excluir nº da lista");
  printf("\n7 - Esvaziar a lista");
  printf("\n8 - Sair");
  printf("\nDigite a opcao: ");
  scanf("%d", &op);

    if (op<1||op>8)
    printf("\nOpcao invalida.\n");
    else
    {
              if (op==1)
              {
                        aux=new(lista);
```

```
            printf("\nDigite o numero a ser inserido: ");
            scanf("%d", &aux->num);
            if (inicio==NULL)
            {
            inicio=aux;
            fim=aux;
            inicio->prox=NULL;
            inicio->ant=NULL;
            }
            else
            {
            aux->prox=inicio;
            inicio->ant=aux;
            aux->ant=NULL;
            inicio=aux;
            }

            printf("\nNumero inserido.\n");
}

if (op==2)
{
            aux=new(lista);
            printf("\nDigite o numero a ser inserido: ");
            std::cin>>aux->num;
            if (inicio==NULL)
```

```
            {
            inicio=aux;
            fim=aux;
            inicio->prox=NULL;
            inicio->ant=NULL;
            }
            else
            {
            fim->prox=aux;
            aux->ant=fim;
            aux->prox=NULL;
            fim=aux;
            }

            printf("\nNumero inserido.\n");
}

if (op==3)
{
            if (inicio==NULL)
            printf("\nLista vazia.\n");
            else
            {
                        aux=inicio;
                        while (aux!=NULL)
                        {
```

```
                    printf("%d\n", aux->num);
                    aux=aux->prox;
                }
            }
}

if (op==4)
{
        if (inicio==NULL)
        printf("\nLista vazia.\n");
        else
        {
                aux=fim;
                while (aux!=NULL)
                {
                        printf("%d\n", aux->num);
                        aux=aux->ant;
                }
        }
}

if (op==5)
{
        if (inicio==NULL)
        printf("\nLista vazia.\n");
        else
```

```
                {
                        qtde=0;
                        printf("\nDigite o nº a ser consultado: ");
                        scanf("%d", &nc);
                        aux=inicio;
                        while (aux!=NULL)
                        {
                                if (aux->num==nc)
                                qtde++;

                                aux=aux->prox;
                        }
                        if (qtde==0)
                        printf("\nNº nao encontrado.\n");
                        else if (qtde==1)
                        printf("\nNúmero %d aparece 1 vez.\n", nc);
                        else
                        printf("\nNúmero %d aparece %d vezes.\n", nc,
qtde);
                }
        }

        if (op==6)
        {
                if (inicio==NULL)
                printf("\nLista vazia.\n");
```

```
else
{
printf("\nDigite o numero que deseja excluir: ");
scanf("%d", &nc);
aux=inicio;
achou=0;
while (aux!=NULL)
{
        if (aux->num==nc)
        {
        achou++;

                if (aux==inicio)
                {
                inicio=inicio->prox;
                inicio->ant=NULL;
                delete(aux);
                aux=inicio;
                }

                else if (aux==fim)
                {
                fim=fim->ant;
                fim->prox=NULL;
                delete(aux);
                aux=NULL;
```

```
                    }

                    else
                    {
                    aux->ant->prox=aux->prox;
                    aux->prox->ant=aux->ant;
                    aux2=aux->prox;
                    delete(aux);
                    aux=aux2;
                    }
                }
                else
                aux=aux->prox;
            }
            if (achou==0)
            printf("\nNumero não encontrado.\n");
            else if (achou==1)
            printf("\nFoi excluída %d ocorrência do nnúmero %d\n",
achou, nc);
            else
            printf("\nForam excluídas %d ocorrências do número
%d\n", achou, nc);
            }
        }


        if (op==7)
```

```
        {
            if (inicio==NULL)
            printf("\nLista vazia.\n");
            else
            {
                aux=inicio;
                while (aux!=NULL)
                {
                    inicio=inicio->prox;
                    delete(aux);
                    aux=inicio;
                }
            printf("\nLista esvaziada.\n");
            }
        }

        }
        system("PAUSE");
    }while (op!=8);
}
```

Lista Duplamente Encadeada e Ordenada

Na estrutura do tipo lista duplamente encadeada e ordenada, cada elemento pode armazenar um ou mais dados (estrutura homogênea ou heterogênea, respectivamente) e dois ponteiros. O primeiro para o próximo elemento, e o segundo para o elemento anterior, permitindo o duplo encadeamento e mantendo

a estrutura linear. Tem-se, também, um campo denominado chave, por meio do qual determinada ordenação é mantida. Segue um exemplo com as operações: inserir na lista, consultar toda a lista do início ao fim ou do fim ao início, consultar um número da lista, remover um elemento qualquer e esvaziá-la.

Exemplo:

```
#include <iostream>
#include <stdio.h>
int main()
{
      struct lista
      {
            int num;
            lista *ant,*prox;

      };
      lista *inicio,*fim,*aux,*aux2;
      inicio=NULL;
      int op,qtde,nc,achou;
   do
   {
     system("CLS");
     printf("\nMENU");
     printf("\n1 - Inserir na lista");
     printf("\n2 - Consultar toda a lista do inicio para o fim");
     printf("\n3 - Consultar toda lista do fim para o inicio");
     printf("\n4 - Consultar um nº da lista");
     printf("\n5 - Excluir nº da lista");
```

```
printf("\n6 - Esvaziar a lista");
printf("\n7 - Sair");
printf("\nDigite a opcao: ");
scanf("%d", &op);

 if (op<1||op>7)
 printf("\nOpcao invalida.\n");
 else
 {
        if (op==1)
        {
                aux=new(lista);
                printf("\nDigite um nº: ");
                scanf("%d", &aux->num);
                if (inicio==NULL)
                {
                inicio=aux;
                fim=aux;
                inicio->prox=NULL;
                inicio->ant=NULL;
                }
                else if (aux->num<=inicio->num)
                {
                        inicio->ant=aux;
                        aux->prox=inicio;
                        aux->ant=NULL;
```

```
                inicio=aux;
        }
        else if (aux->num>=fim->num)
        {
                fim->prox=aux;
                aux->ant=fim;
                aux->prox=NULL;
                fim=aux;
        }
        else
        {
                aux2=inicio;
                while (aux2->prox->num < aux->num)
                aux2=aux2->prox;

                aux->ant=aux2;
                aux->prox=aux2->prox;
                aux2->prox=aux;
                aux->prox->ant=aux;
        }

        printf("\nNumero inserido.\n");
}
if (op==2)
{
        if (inicio==NULL)
```

```
            printf("\nLista vazia.\n");
            else
            {
                  aux=inicio;
                  while (aux!=NULL)
                  {
                        printf("%d\n", aux->num);
                        aux=aux->prox;
                  }
            }
}

if (op==3)
{
            if (inicio==NULL)
            printf("\nLista vazia.\n");
            else
            {
                  aux=fim;
                  while (aux!=NULL)
                  {
                  printf("%d\n", aux->num);
                  aux=aux->ant;
                  }
            }
}
```

```
if (op==4)
{
        if (inicio==NULL)
        printf("\nLista vazia.\n");
        else
        {
                qtde=0;
                printf("\nDigite o nº a ser consultado: ");
                scanf("%d", &nc);
                aux=inicio;
                while (aux!=NULL && aux->num<=nc)
                {
                        if (aux->num==nc)
                        qtde++;

                        aux=aux->prox;
                }
                if (qtde==0)
                printf("\nNº nao encontrado.\n");
                else if (qtde==1)
                printf("\nNúmero %d aparece 1 vez.\n", nc);
                else
                printf("\nNúmero %d aparece %d vezes.\n", nc, qtde);
        }
}
```

```
if (op==5)
{
        if (inicio==NULL)
        printf("\nLista vazia.\n");
        else
        {
        printf("\nDigite o numero que deseja excluir: ");
        scanf("%d", &nc);
        aux=inicio;
        achou=0;
        while (aux!=NULL && aux->num<=nc)
        {
                if (aux->num==nc)
                {
                achou++;

                        if (aux==inicio)
                        {
                        inicio=inicio->prox;
                        inicio->ant=NULL;
                        delete(aux);
                        aux=inicio;
                        }

                        else if (aux==fim)
```

```
                    {
                    fim=fim->ant;
                    fim->prox=NULL;
                    delete(aux);
                    aux=NULL;
                    }

                    else
                    {
                    aux->ant->prox=aux->prox;
                    aux->prox->ant=aux->ant;
                    aux2=aux->prox;
                    delete(aux);
                    aux=aux2;
                    }
                }
                else
                aux=aux->prox;
            }
            if (achou==0)
            printf("\nNumero não encontrado.\n");
            else if (achou==1)
                printf("\nFoi excluída %d ocorrência do nnúmero %d\n",
achou, nc);
            else
                printf("\nForam excluídas %d ocorrências do número
%d\n", achou, nc);
```

```
                }
        }

        if (op==6)
        {
                if (inicio==NULL)
                printf("\nLista vazia.\n");
                else
                {
                        aux=inicio;
                        while (aux!=NULL)
                        {
                                inicio=inicio->prox;
                                delete(aux);
                                aux=inicio;
                        }
                printf("\nLista esvaziada.\n");
                }
        }

    }
    system("PAUSE");
  }while (op!=7);
}
```



ACOMPANHE NA WEB

Listas Encadeadas: Conceito, Representação e Algoritmos

Acesse o texto Listas Encadeadas: Conceito, Representação e Algoritmos. Entenda o conceito de listas encadeadas, conheça sua representação usual; entenda a diferença entre alocação sequencial e alocação encadeada de memória no contexto do armazenamento de conjuntos de elementos.

Disponível em: <http://www2.dc.ufscar.br/~bsi/materiais/ed/u6.html>. Acesso em: 25 jun. 2014.

Conceito de Lista Linear Simplesmente Encadeada

Assista ao vídeo Conceito de Lista Linear Simplesmente Encadeada. Saiba mais sobre alocação estática e dinâmica. Veja um exemplo de inserção no início de uma lista linear simplesmente encadeada.

Disponível em: <http://homepages.dcc.ufmg.br/~rodolfo/aedsi-2-10/Funcoes/funcoes.html>. Acesso em: 1 jul. 2014.

Tempo: 12min59seg.

Inserir no Fim de uma Lista Linear Simplesmente Encadeada

Assista ao vídeo Inserir no Fim de uma Lista Linear Simplesmente Encadeada. Este vídeo aborda o conceito de lista linear simplesmente encadeada e apresenta um exemplo de inserção no fim de uma lista linear simplesmente encadeada.

Disponível em: <https://www.youtube.com/watch?v=-xzfTsafixI>. Acesso em: 25 jun. 14.

Tempo: 12min17seg.



AGORA É A SUA VEZ

Instruções:

Agora, chegou a sua vez de exercitar seu aprendizado. A seguir, você encontrará algumas questões de múltipla escolha e dissertativas. Leia cuidadosamente os

enunciados e atente-se para o que está sendo pedido.

Questão 1

Com o intuito de verificar as habilidades de programação adquiridas até agora, a atividade proposta é a solução de um problema simples utilizando os recursos de programação estudados no tema anterior. Tais conhecimentos são extremamente necessários na sequência dos conteúdos, pois teremos muitos exemplos e exercícios fundamentados na linguagem C.

Problema proposto: Implementar um programa em C que carregue uma fila dinâmica homogênea com números inteiros positivos. O programa deverá executar as operações a seguir:

a) Inserir número na fila.

b) Listar toda a fila.

c) Excluir um número da fila.

d) Esvaziar a fila.

Questão 2

Observe o código, a seguir, e classifique o tipo de lista utilizada com relação à estrutura de programação:

```
aux=new(lista);

std::cin>>aux->num;

aux2->prox=aux;

aux->ant=aux2;

aux->prox=NULL;

aux2=aux;
```

a) Simplesmente encadeada com inserção no início da lista.

b) Duplamente encadeada com inserção no fim da lista.

c) Simplesmente encadeada com inserção no fim da lista.

d) Duplamente encadeada com inserção no início da lista.

e) Nenhuma das alternativas é verdadeira.

Questão 3

Observe o código, a seguir, e classifique o tipo de lista utilizada com relação à estrutura de programação:

```
aux=new(lista);

std::cin>>aux->num;

if (aux1==NULL)

aux2=aux;

aux->prox=aux1;

aux1=aux;
```

a) Simplesmente encadeada com inserção no início da lista.

b) Simplesmente encadeada com inserção no fim da lista.

c) Duplamente encadeada com inserção no fim da lista.

d) Duplamente encadeada com inserção no início da lista.

e) Nenhuma das alternativas é verdadeira.

Questão 4

Com base na estrutura de programação da linguagem C, crie um programa que carregue uma lista simplesmente encadeada e não ordenada para simular um controle de voos. A lista deverá manipular uma estrutura contendo o número do voo, o número de lugares disponíveis no voo, a origem e o destino do voo. Resolva os itens a seguir:

a) Cadastrar voo.

b) Consultar por número do voo.

c) Consultar todos os voos com lugares disponíveis.

d) Excluir um voo.

e) Esvaziar a lista de voos

Questão 5

Com base na estrutura de programação da linguagem C, crie um programa que carregue uma lista simplesmente encadeada e ordenada para simular um sistema de controle de alunos. A lista deverá manipular uma estrutura contendo o número,

o nome, a nota na prova 1 e a nota na prova 2. Resolva os itens a seguir:

a) Inserir aluno ordenado pelo número.

b) Consultar por inicial do nome.

c) Consultar os aprovados.

d) Mostrar a quantidade de reprovados.

e) Excluir aluno.

f) Mostrar todos os alunos com média e situação.

Observação: é considerado aprovado o aluno com média aritmética maior ou igual a 6.



FINALIZANDO

Os conceitos e exemplos práticos em C apresentados neste tema deixaram bem claro que a estrutura de dados do tipo lista é muito versátil na solução de problema do cotidiano. Sua estrutura permite armazenar tipos de dados variados, possibilitando o trabalho com estruturas homogêneas e heterogêneas.

As operações básicas de inserção, consultar toda a lista, consultar lista do início para o fim e do fim para o início, remoção e esvaziamento da lista foram bem exploradas nos exemplos e exercícios propostos, bem como o foram as estruturas de lista ordenada, não ordenada, homogênea e heterogênea com exemplos em C para cada uma das estruturas.



REFERÊNCIAS

LISTAS Encadeadas: Conceito, Representação e Algoritmos. Disponível em: <http://www2.dc.ufscar.br/~bsi/materiais/ed/u6.html>. Acesso em: 17/06/2014.

CONCEITO de Lista Linear Simplesmente Encadeada. Disponível em: <https://www.youtube.com/watch?v=5FG7FC-6Fng>. Acesso em: 17/06/2014.

CONCEITO de Lista Linear Simplesmente Encadeada. Disponível em: <https://www.youtube.com/watch?v=-xzfTsafixI>. Acesso em: 17/06/2014.

TENENBAUM, Aaron M.; LANGSAM, Yedidyah; AUGENSTEIN, Moshe J. Estruturas de dados usando C. São Paulo: Makron Books, 1995.



GLOSSÁRIO

Encadeada: ligar em cadeia, estar numa sequência, ter conexão.

Estrutura de Dados: na Computação, estrutura de dados é uma forma específica de organização e armazenamento de dados, para que sejam utilizados de forma eficaz. As estruturas de dados e seus algoritmos são muito utilizados na Ciência da Computação em diversas áreas do conhecimento e com as mais diferentes finalidades na solução de problemas computacionais.

Heterogênea: aquilo que é composto por elementos diferentes, desiguais.

Homogênea: cujas partes são da mesma natureza ou estão estreitamente ligadas. Idêntica, igual, análoga.

Árvores binárias



POR DENTRO DO TEMA

Uma árvore binária é um conjunto finito de elementos, em que cada elemento é denominado nó, e o primeiro nó é chamado de raiz da árvore.

Esse conjunto pode estar vazio ou ser dividido em três subconjuntos distintos, sendo eles: 1º subconjunto (nó raiz), 2º subconjunto (subárvore direita) e 3º subconjunto (subárvore esquerda).

Com relação às operações em árvores binárias, serão abordadas as seguintes: inserir um nó na árvore, removê-lo, consultar os nós da árvore em ordem, consultar em pré-ordem, consultar em pós-ordem e esvaziar a árvore.

Na operação de inserção, as propriedades de uma árvore devem ser obedecidas, e todo novo nó é sempre uma folha. Na operação de remoção, o filho da direita, que é o mais velho, assume o lugar do nó pai.

Nas operações de consulta, em ordem, pré-ordem e pós-ordem, todos os nós da árvore são listados, alterando-se apenas sua ordem. Na consulta em ordem, cada árvore é mostrada com o ramo da esquerda, a raiz e, posteriormente, o ramo da direita. Na consulta pré-ordem, cada árvore é mostrada com a raiz, o ramo da esquerda e, posteriormente, o ramo da direita. Na consulta pós-ordem, cada árvore é mostrada com o ramo da esquerda, o ramo da direita e, posteriormente, a raiz.

A relação existente entre a altura da árvore (h) e o número de nós (n) de uma árvore binária é uma informação muito importante em muitas aplicações. É comum a pergunta pela altura máxima e mínima de árvores binárias. Possuem altura máxima aquelas em que cada nó possui apenas um único filho. A altura de tais árvores é igual a n. Já uma árvore completa possui altura mínima.

A operação de busca em uma árvore binária é igual ao número de nós existentes no caminho, desde a raiz da árvore até o nó procurado. Em uma árvore binária genérica, no pior caso, esse nó se encontra a uma distância O(n) da raiz da árvore; logo, a complexidade da busca é O(n). Conclui-se, então, que a complexidade de

busca corresponde à altura da árvore. No melhor caso, em que uma árvore pode possuir altura mínima, que é o caso de uma árvore binária completa, o tempo de busca é O(log n).

Considerando, ainda, uma árvore de altura mínima, na operação de inserção, o nó sempre é inserido em uma folha, tendo de percorrer todos os nós, desde a raiz, até chegar em uma folha e acrescentar um filho a ela, gastando com isso a altura da árvore, ou seja, O(log n).

Na operação de remoção, o pior caso acontece quando o nó a ser removido encontra-se em uma folha no nível mais baixo. Gasta-se a altura da árvore para encontrá-lo, no caso de uma árvore de altura mínima, e algumas operações constantes de atualização de **ponteiros**, gerando uma complexidade O(log n).

Árvore Binária Recursiva

Nesta estrutura, cada elemento armazena um tipo de dado e ponteiros para o elemento à esquerda e à direita, o que permite a inserção dos valores na árvore de forma recursiva.

Segue um exemplo com as seguintes operações: inserir um número, consultar um número, consultar toda a árvore em ordem, consultar toda a árvore em pré-ordem, consultar toda a árvore em pós-ordem, excluir um número da árvore.

Exemplos de Árvore Binária em C

```
#include <iostream>
#include <stdio.h>
struct arvore
{
   int num;
   arvore *direita, *esquerda;
};
arvore *raiz, *aux;
arvore *insere_R(arvore *aux, int num)
{
  if (aux == NULL)
    {
```

```
    aux = new (arvore);
    aux->num = num;
    aux->esquerda = NULL;
    aux->direita = NULL;
   }
  else if (num < aux->num)
        aux->esquerda=insere_R(aux->esquerda, num);
       else
        aux->direita=insere_R(aux->direita, num);
  return aux;
}
int consultar(arvore *aux, int num, int achou)
{
  if (aux != NULL && achou == 0)
  {
  if (aux->num == num)
    {
      achou = 1;
    }
  else if (num < aux->num)
        achou = consultar(aux->esquerda, num, achou);
       else
        achou = consultar(aux->direita, num, achou);
  }
  return achou;
}
```

```
void mostrarordem (arvore *aux)
{
  if (aux != NULL)
        {
         mostrarordem(aux->esquerda);
         printf("\n%d\n", aux -> num);
         mostrarordem(aux->direita);
        }
}
void mostrarpre (arvore *aux)
{
  if (aux != NULL)
        {
         printf("\n%d\n", aux -> num);
         mostrarpre(aux->esquerda);
         mostrarpre(aux->direita);
        }
}
void mostrarpos (arvore *aux)
{
  if (aux != NULL)
        {
         mostrarpos(aux->esquerda);
         mostrarpos(aux->direita);
         printf("\n%d\n", aux -> num);
        }
```

```
}
arvore *remove(arvore *aux, int num)
{
  arvore *p, *p2;
  if (aux->num == num)
   {
    if (aux->esquerda == aux->direita)
     {
       delete(aux);
       return NULL;
     }
    else if (aux->esquerda == NULL)
         {
           p=aux->direita;
           delete(aux);
           return p;
          }
         else if (aux->direita == NULL)
                  {
                    p=aux->esquerda;
                    delete(aux);
                    return p;
                   }
                  else
                   {
                     p2= aux->direita;
```

```
                    p= aux->direita;
                    while (p->esquerda)
                     {
                       p=p->esquerda;
                     }
                     p->esquerda = aux->esquerda;
                     delete(aux);
                     return p2;
                   }
   }
   if (aux->num < num)
    aux->direita = remove(aux->direita, num);
   else
    aux->esquerda = remove(aux->esquerda, num);
   return aux;
}
int main()
{
   int op, achou, numero;
   raiz = NULL;
   do
   {
    system("CLS");
    printf("\nMenu de opções");
    printf("\n1 - Inserir um número na árvore");
    printf("\n2 - Consultar um número da árvore");
```

```
printf("\n3 - Consultar toda a árvore em ordem");
printf("\n4 - Consultar toda a árvore em pré-ordem");
printf("\n5 - Consultar toda a árvore em pós-ordem");
printf("\n6 - Excluir um número da árvore");
printf("\n7 - Sair");
printf("\nDigite sua opção: ");
scanf("%d", &op);
if (op < 1 || op > 7)
 printf("\n\nOpção inválida.\n");
if (op == 1)
 {
   printf("\nDigite o n£mero a ser inserido: ");
   scanf("%d", &numero);
   raiz=insere_R(raiz,numero);
   printf("\nNúmero inserido.\n");
 }
if (op == 2)
 {
    printf("\nDigite o número a ser consultado: ");
    scanf("%d", &numero);
   achou = consultar(raiz,numero,0);
   if (achou == 0)
       printf("\nNúmero não encontrado.\n");
   else
       printf("\nNúmero encontrado.\n");
 }
```

```
if (op == 3)
 {
  if (raiz == NULL)
        printf("Árvore vazia.\n");
  else
   {
        aux = raiz;
        mostrarordem(aux);
   }
 }
if (op == 4)
 {
  if (raiz == NULL)
    printf("Árvore vazia.\n");
  else
   {
    aux = raiz;
    mostrarpre(aux);
       }
 }
if (op == 5)
 {
  if (raiz == NULL)
     printf("Árvore vazia.\n");
  else
   {
```

```
        aux = raiz;
        mostrarpos(aux);
      }
    }
   if (op == 6)
    {
     if (raiz == NULL)
       printf("Árvore vazia.\n");
     else
      {
        printf("\nDigite o número a ser removido: ");
        scanf("%d", &numero);
       achou = consultar(raiz,numero,0);
       if (achou == 0)
        printf("\nNúmero não encontrad.\n");
       else
        {
          raiz=remove(raiz,numero);
          printf("\nNúmero removido.\n");
        }
      }
    }
    system("PAUSE");
   }
   while (op!=7);
}
```

Árvore AVL

A árvore AVL é uma árvore binária balanceada, ou seja, é uma árvore que obedece a todas as propriedades da árvore binária e em que cada nó apresenta diferença de altura entre as subárvores direita e esquerda de 1, 0 ou –1.

Se a diferença de altura entre as subárvores de um nó é maior que 1 ou menor que –1, a árvore está desbalanceada e haverá uma rotação.

Nas árvores balanceadas, o custo das operações se mantém na mesma ordem de grandeza das árvores binárias, como a árvore AVL, por exemplo.

Na estrutura do tipo árvore AVL, cada nó armazena a altura da sua subárvore esquerda e direita, o que facilita as operações de inserção e remoção e torna o custo dessas operações na ordem de O(log n).

Árvore AVL com Balanceamento

Nesta estrutura, cada elemento armazena um número e variáveis para controlar a altura direita e esquerda, além de ponteiros para o elemento à esquerda e à direita, o que permite a inserção dos valores na árvore.

Segue um exemplo com as seguintes operações: inserir, mostrar e excluir um número da árvore AVL. A cada inserção de um novo número ou exclusão, caso seja necessário, é feita a rotação para manter o **balanceamento da árvore**.

Exemplos de Árvore AVL em C

```
#include <iostream>
#include <stdio.h>
struct AVL
{
  int num, altd, alte;
  AVL *dir, *esq;
};
AVL *rotacao_esquerda(AVL *pont)
{
 AVL *aux1, *aux2;
 aux1 = pont -> dir;
```

```
  aux2 = aux1 -> esq;
  pont -> dir = aux2;
  aux1 -> esq = pont;
  if (pont->dir == NULL)
       pont->altd = 0;
  else if (pont->dir->alte > pont ->dir->altd)
                  pont -> altd = pont->dir->alte+1;
          else
                  pont -> altd = pont->dir->altd+1;
  if(aux1->esq->alte > aux1->esq->altd)
       aux1 -> alte = aux1 ->esq-> alte + 1;
  else
       aux1 -> alte = aux1 ->esq-> altd + 1;
  return aux1;
}
AVL *rotacao_direita(AVL *pont)
{
  AVL *aux1, *aux2;

  aux1 = pont -> esq;
  aux2 = aux1 -> dir;
  pont -> esq = aux2;
  aux1 -> dir = pont;
       if (pont->esq == NULL)
       pont->alte = 0;
  else if (pont->esq->alte > pont ->esq->altd)
```

```
                pont -> alte = pont->esq->alte+1;
        else
                pont -> alte = pont->esq->altd+1;
 if(aux1->dir->alte > aux1->dir->altd)
      aux1 -> altd = aux1 ->dir-> alte + 1;
 else
      aux1 -> altd = aux1 ->dir->altd + 1;
 return aux1;
}
AVL *balanceamento(AVL *pont)
{
 int d, df;
 d = pont -> altd - pont -> alte;
 if (d == 2)
  {
      printf("\nentrou para balancear com d = 2.\n");
      df = pont->dir->altd - pont->dir->alte;
      if (df == 1)
       {
           printf("\nentrou para balancear com df = 1.\n");
        pont = rotacao_esquerda(pont);
       }
      else
       {
                printf("\nentrou para balancear com df = -1.\n");
          pont->dir = rotacao_direita(pont->dir);
```

```
        pont = rotacao_esquerda(pont);
        }
  }
  else if (d == -2)
        {
                        printf("\nentrou para balancear com d = -2.\n");
           df = pont->esq->altd - pont->esq->alte;
              if (df == -1)
               {
                              printf("\nentrou para balancear com df = -1.\n");
                    pont = rotacao_direita(pont);
          }
             else
             {
                      printf("\nentrou para balancear com df = 1.\n");
                pont->esq = rotacao_esquerda(pont->esq);
                pont = rotacao_direita(pont);
           }
        }
     return pont;
}
void mostrar(AVL *pont, int alt)
{
      if (pont != NULL)
      {
     mostrar(pont->esq, alt+1);
```

```
            printf("\n%d \taltura= %d \tpontesq= %d \tpontdir= %d\n", pont->num, alt,
pont->alte, pont->altd);
             mostrar(pont->dir, alt+1);
        }
    }
    void buscar_toda(AVL *pont)
    {
        if (pont != NULL)
        {
       buscar_toda(pont->esq);
       buscar_toda(pont->dir);
       pont = balanceamento(pont);
        }
    }
    AVL *inserir(AVL *pont, int n)
    {
      AVL *novo;
      if (pont == NULL)
       {
          novo = new(AVL);
          novo->num = n;
          novo->altd = 0;
          novo->alte = 0;
          novo->dir = NULL;
          novo->esq = NULL;
          pont = novo;
```

```
        printf("\nNúmero inserido.\n");
    }
    else if (n < pont -> num)
        {
         pont->esq = inserir(pont->esq,n);
         if (pont->esq->altd > pont->esq->alte)
                    pont ->alte = pont->esq->altd + 1;
         else
                    pont ->alte = pont->esq->alte + 1;
         printf("\n Mostrar antes do balanceamento");
         mostrar(pont,0);
         pont = balanceamento(pont);
        }
          else
        {
         pont->dir = inserir(pont->dir,n);
         if (pont->dir->altd > pont->dir->alte)
                    pont ->altd = pont->dir->altd + 1;
         else
                    pont ->altd = pont->dir->alte + 1;
         printf("\n Mostrar antes do balanceamento");
         mostrar(pont,0);
         pont = balanceamento(pont);
     }
     return pont;
}
```

```
AVL *buscar(AVL *pont, int n)
{
  int achou = 0;
  while (pont != NULL && achou == 0)
  {
      if (pont -> num == n)
       achou = 1;
      else if (pont -> num > n)
                pont = pont -> esq;
               else
                pont = pont -> dir;
  }
  return pont;
}
void balanceamento_pai(AVL *pont, AVL *filho)
{
 if (pont->dir == filho)
      pont ->altd = pont->altd - 1;
  else
   pont ->alte = pont->alte - 1;
  printf("\n pai= %d      altura esq.= %d      altura dir.= %d\n", pont->num, pont-
>alte, pont->altd);
}
AVL *buscar_pai(AVL *pai, AVL *filho, int el)
{
   while (pai -> esq != filho && pai ->dir != filho)
```

```
        {
            if (pai -> num > el)
              pai = pai -> esq;
          else
              pai = pai -> dir;
        }
    return pai;
}
AVL *remover(AVL *pont, int el, AVL *r)
{
    AVL *p, *p2, *aux;
        if (pont->num == el)
        {
        if (pont->esq == pont->dir)
                {
          aux = buscar_pai(r,pont,el);
          if (aux != NULL)
                        {
                                balanceamento_pai(aux,pont);
                        }
                        delete(pont);
                        return NULL;
     }
          else if (pont->esq == NULL)
                        {
            p=pont->dir;
```

```
                aux = buscar_pai(r,pont,el);
                if (aux != NULL)
                {
                                balanceamento_pai(aux,pont);
                }
                delete(pont);
                return p;
              }
              else if (pont->dir == NULL)
                        {
p=pont->esq;
                                aux = buscar_pai(r,pont,el);
                                if (aux != NULL)
                                {
                                        balanceamento_pai(aux,pont);
                                }
                                delete(pont);
                                return p;
                        }
                        else
                        {
 p2= pont->dir;
                                 p= pont->dir;
                                 while (p->esq)
                                 {
      p=p->esq;
```

```
                                        }
                                        p->esq = pont->esq;
                                        if (pont->altd >pont->alte)
                                                p2 ->alte = p2->alte + pont->altd;
                                        else
                                                p2 ->alte = p2->alte + pont->alte;
                                        aux = buscar_pai(r,p,el);
                                        if (aux != NULL)
                                        {
                                                balanceamento_pai(aux,pont);
                                        }
                                        delete(pont);
                                       return p2;
        }


            }
            if (pont->num < el)
        {
                pont->dir = remover(pont->dir, el,r);
            }
            else
{
                        pont->esq = remover(pont->esq, el,r);
            }
            pont = balanceamento(pont);
            return pont;
```

```
}
int main()
{
    int op,n;
    AVL *raiz, *p;
    raiz = NULL;
    do
    {
        system("CLS");
        printf("\nMenu");
        printf("\n1 - Inserir");
        printf("\n2 - Mostrar");
        printf("\n3 - Excluir");
        printf("\n4 - Sair");
        printf("\nDigite sua opção: ");
        scanf("%d", &op);
        if (op < 1 || op > 4)
        printf("\nOpção inválida.\n");
        if (op == 1)
        {
                printf("\nDigite o número a ser inserido: ");
                scanf("%d", &n);
                raiz = inserir(raiz,n);
        }
        if (op == 2)
        {
```

```
                mostrar(raiz,0);
        }
        if (op == 3)
        {
                printf("\nDigite o número a ser excluído: ");
                scanf("%d", &n);
                p = buscar(raiz,n);
                if (p == NULL)
                 printf("\nNúmero não encontrado.\n");
                else
                 {
                  raiz=remover(raiz,n,raiz);
                  buscar_toda(raiz);
                  printf("\n Número excluído.\n");
                 }
        }
        system("PAUSE");
    }while (op !=4);
}
```



ACOMPANHE NA WEB

Árvores binárias

Uma árvore binária é uma estrutura de dados mais geral que uma lista encadeada. O link indicado, a seguir, introduz as operações mais simples sobre árvores binárias:

Disponível em: <http://www.ime.usp.br/~pf/algoritmos/aulas/bint.html>. Acesso em: set. 2014.

Árvores AVL (Balanceadas)

Uma árvore binária balanceada (AVL) é uma árvore binária na qual as alturas das duas subárvores de todo nó nunca diferem em mais de 1. Acesse o material "Árvores AVL (Balanceadas)".

Disponível em: <http://wiki.icmc.usp.br/images/f/f0/AVL.pdf>. Acesso em: set. 2014.

Programar em C - 08 - Árvore Binária de Busca

Assista ao vídeo Programar em C - 08 - Árvore Binária de Busca. Programa em C que implementa árvore binária de busca.

Disponível em: <http://www.youtube.com/watch?v=FqDVZ8SHBto>. Acesso em: set. 2014. Tempo: 19min21seg

Árvores Balanceadas – AVL

Assista ao vídeo Árvores Balanceadas – AVL. Uma árvore é considerada AVL se, e somente se, para cada um de seus nós, as alturas das subárvores à direita e à esquerda forem iguais ou difiram em apenas uma unidade.

Disponível em: <http://www.youtube.com/watch?v=xNyQ6F_1SbE>. Acesso em: set. 2014. Tempo: 10min36seg



AGORA É A SUA VEZ

Instruções:

Agora, chegou a sua vez de exercitar seu aprendizado. A seguir, você encontrará algumas questões de múltipla escolha e dissertativas. Leia cuidadosamente os enunciados e atente-se para o que está sendo pedido.

Questão 1

Com o intuito de verificar as habilidades de programação adquiridas até agora, a atividade proposta é a solução de um problema simples utilizando os recursos de programação estudados no tema anterior. Tais conhecimentos são extremamente necessários na sequência dos conteúdos, pois teremos muitos exemplos e

exercícios fundamentados na linguagem C.

Problema proposto: Com base na estrutura de programação da linguagem C, criar um programa que carregue uma lista simplesmente encadeada e ordenada para simular um sistema de controle de notas. A lista deverá manipular uma estrutura contendo o número e a média dos alunos. Resolver os itens a seguir:

1) Inserir número ordenado.

2) Consultar os aprovados.

3) Consultar os reprovados.

4) Mostrar a quantidade de aprovados.

5) Mostrar a quantidade de reprovados.

Observação: é considerado aprovado o aluno com média maior ou igual a 6.

Questão 2

Assinale a alternativa incorreta com relação às propriedades da árvore binária.

a) Todos os nós de uma subárvore direita são maiores que o nó raiz.

b) O nó pai sempre deve ser maior que um nó filho.

c) Cada subárvore é também uma árvore binária.

d) Todos os nós de uma subárvore esquerda são menores que o nó raiz.

e) Nenhuma das alternativas é verdadeira.

Questão 3

Entre as alternativas, qual satisfaz uma característica de árvore binária:

a) Árvore estritamente binária é a árvore em que todos os nós têm 0 ou mais filhos.

b) Nó pai é o nó acima e com ligação indireta a outro nó.

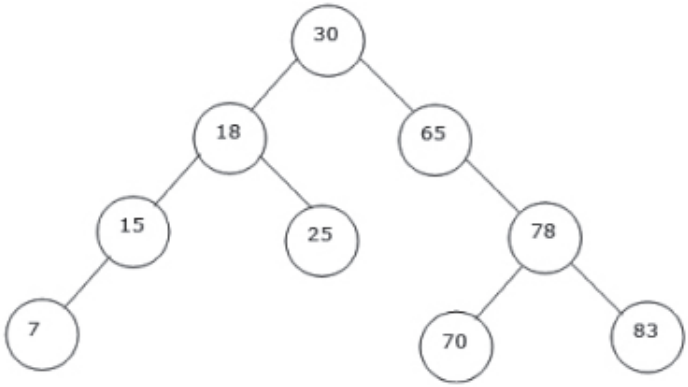
c) Uma árvore binária tem grau máximo igual a 4.

d) Nós irmãos são os nós que possuem o mesmo nó pai.

e) Nenhuma das alternativas é verdadeira.

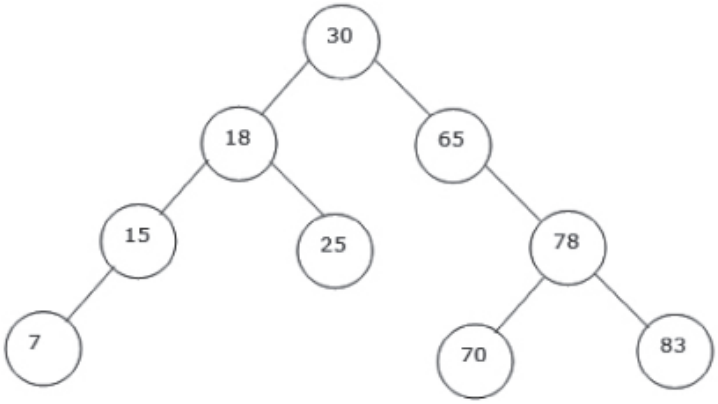
Questão 4

A árvore, a seguir, é estritamente binária? Justifique sua resposta.



Questão 5

A árvore, a seguir, pode ser considerada completa? Justifique sua resposta.





FINALIZANDO

Os conceitos e exemplos práticos em C apresentados neste tema deixaram bem claro que a estrutura de dados do tipo árvore binária e árvore AVL é muito versátil na solução de problemas do cotidiano.

Na estrutura do tipo árvore, cada elemento armazena um tipo de dado e ponteiros para o elemento à esquerda e à direita, o que permite a inserção dos valores na árvore de forma recursiva.

As operações básicas de inserção, consulta, consultar toda a árvore em ordem, consultar toda a árvore em pré-ordem, consultar toda a árvore em pós-ordem, excluir um número da árvore foram abordadas com exemplos práticos.



REFERÊNCIAS

ARAÚJO, Renato Paschoal de. Árvores Balanceadas – AVL. Disponível em: <http://www.youtube.com/watch?v=xNyQ6F_1SbE>. Acesso em: 22/06/2014.

FEOFILOFF, Paulo. Árvores binárias. Disponível em: <http://www.ime.usp.br/~pf/algoritmos/aulas/bint.html>. Acesso em: 22/06/2014.

GOMES, Italo. Árvore Binária de Busca. Disponível em: <http://www.youtube.com/watch?v=FqDVZ8SHBto>. Acesso em: 22/06/2014.

ROMERO, Roseli Ap. Francelin. Árvore AVL (balanceada). Disponível em: <http://wiki.icmc.usp.br/images/f/f0/AVL.pdf>. Acesso em: 22/06/2014.

TENENBAUM, Aaron M.; LANGSAM, Yedidyah; AUGENSTEIN, Moshe J. Estruturas de dados usando C. São Paulo: Makron Books, 1995.



GLOSSÁRIO

Balanceamento da Árvore: é a operação de ajuste das alturas das subárvores direita e esquerda que não pode ser mais que 1. O balanceamento ocorre realizando-se a rotação simples ou dupla.

Ponteiro: por definição, um ponteiro é uma variável que armazena o endereço de outra variável, ou seja, enquanto uma variável do tipo inteiro armazena valores inteiros, um ponteiro armazena o endereço de memória no qual outra variável está alocada, e não seu valor.

Árvore multidirecional (Árvore B)



POR DENTRO DO TEMA

A árvore B ou B-Tree é muito estudada na computação, sendo esta uma estrutura de dados bastante utilzada nos sistemas de arquivos e nos bancos de dados.

Nas operações de inserção e remoção de dados, o nó não pode ser maior que sua ordem e, também, não pode ser menor que sua ordem dividida por 2.

Diferentemente das árvores de busca binária e AVL, a árvore B não precisa ser rebalanceada. Outra característica importante da árvore B, que aponta vantagens se comparada a outros tipos de árvores, é o menor tempo de acesso e pesquisa.

Para uma árvore B de ordem n, onde n representa o máximo de filhos para cada nó, só é uma árvore B se atender às seguintes propriedades:

1. Cada nó pode ter no máximo n filhos.
2. Cada nó, exceto o nó raiz e os nós folhas, deve ter pelo menos n/2 filhos.
3. O nó raiz deve ter pelo menos dois filhos, a menos que seja um nó folha.
4. Todo nó não-folha com m filhos deverá ter m-1 chaves.
5. Folhas têm que aparecer no mesmo nível e armazenam dados.

A representação dos nós em árvore B denota conjuntos de elementos com apontadores para seus filhos, denominados folhas. Alguns autores definem como ordem de uma árvore B a quantidade de registros que a árvore pode suportar, porém, há autores que consideram a ordem de uma árvore B a quantidade de campos apontadores.

Os nós de uma árvore B devem ter um número mínimo de elementos, que é dado calculando a metade do valor da ordem do nó. Caso a árvore seja de ordem ímpar, o número deve ser truncado, com exceção da raiz, que pode ter um registro. Por exemplo, para uma árvore de ordem 7, deve-se ter 7 / 2 = 3,5 registros no mínimo; truncando, teremos 3 registros.

Exemplo de estrutura em C:

```
#define N 5 // definindo a ordem da árvore

struct No{

   int num;

   char chave[N-1];

   struct No *prox[N];

};
```

Inserção:

1. Pesquisar se a chave não existe na árvore.
2. Buscar a posição para inserir o valor. Testar o nó para verificar se está cheio.
3. Caso o nó esteja vazio, inserir o valor, caso contrário, deve-se executar a subdivisão do nó:

1. Verificar se o nó-pai está vazio, se sim:

2. Mover para o nó pai o elemento do meio.

3. Dividir em dois nós iguais.

2. Se o nó pai não estiver vazio, repetir recursivamente os passos acima. Caso todos os nós-pai estejam cheios, incluindo o nó raiz, uma nova raiz terá que ser criada, aumentando a altura da árvore.

4. A chave só será inserida após satisfazer todas as divisões possíveis.

Exclusão:

1. Pesquisar a chave para verificar se existe.
2. Se existir e estiver em uma folha, fazer a exclusão.
3. Se existir e não estiver em uma folha, substituir a chave pela menor chave do filho à sua direita.

1. Se o número da chave no nó for maior do que N/2-1 (N é a ordem da árvore), finalizar.

2. Caso contrário, redistribuir as chaves entre os nós vizinhos.

Busca:

1. Receber a chave a ser pesquisada.
2. Pesquisar começando da raiz até encontrar a chave e retornar o nó e a posição da chave na árvore.
3. Se a chave não for encontrada, continue o laço até varrer as folhas.

Vantagens:

1. Por ter um número menor de nós do que uma árvore binária, possui melhor desempenho. A quantidade menor de nós indica uma árvore de menor altura, resultando em menos acessos ao disco.
2. Como tem poucos ponteiros entre os nós, utiliza menos espaço de alocação.
3. A utilização de chaves primárias garante maior rapidez nas buscas.
4. A cada operação, inclusão ou exclusão, a estrutura dinâmica faz o balanceamento automático da árvore.
5. Nas buscas aleatórias, devido às ramificações, possibilita um tempo menor de acesso aos dados.

Desvantagens:

1. A busca às vezes pode ser lenta, pois um nó não-folha com n chaves é visitado n vezes.

Comparações com outras Estruturas de Dados

Inserção: Mais rápida em comparação com **tabelas hash**, devido aos balanceamentos a cada inserção, ocorrendo menos colisões e repetições para encontrar posições livres.

Exclusão: Nas operações de exclusão, as árvores B têm a característica de evitar grandes fragmentações, se ajustando e balanceando. Quanto às tabelas hash, se acontecer a exclusão de uma chave, a alocação de memória permanece vazia, entretanto, o espaço continua alocado.

Busca: A tulização de chave primária é o fator que faz a diferença no desempenho superior da árvore B. A busca se inicia na raiz e precorre os nós e filhos conforme a chave procurada, tornando o resultado final da busca mais eficiente.

1º exemplo: algoritmo, trecho de programa em C

```
const
```

```
    T = 2,
    MAX_CHAVES = 2 * T - 1, //Quantidade máxima de chaves
    MAX_FILHOS = 2 * T, //Quantidade máxima de filhos
    MIN_OCUP = T - 1; //Ocupação mínima em cada nó
typedef struct no_arvoreB arvoreB;
struct no_arvoreB {
  int num_chaves; //Quantidades de chaves contida no nó
  int chaves[MAX_CHAVES]; //Chaves armazenadas no nó
  arvoreB *filhos[MAX_FILHOS]; //Ponteiro para os filhos
};

//inserir uma chave e o ponteiro para o filho da direita em um nó
void insere_chave(arvoreB *raiz, int info, arvoreB *filhodir)
{
 int k, pos;
 //realiza busca para obter a posição ideal para inserir a nova chave
 pos = busca_binaria(raiz, info);
 k = raiz->num_chaves;
 //remanejamento para manter as chaves ordenadas
 while (k > pos && info < raiz->chaves[k-1])
 {
  raiz->chaves[k] = raiz->chaves[k-1];
  raiz->filhos[k+1] = raiz->filhos[k];
  k--;
 }
 //inserir chave na posição ideal
```

```
  raiz->chaves[pos] = info;
  raiz->filhos[pos+1] = filhodir;
  raiz->num_chaves++;
}
//busca do nó para inserir a chave e subdivisões quando necessárias
arvoreB *insere(arvoreB *raiz, int info, bool *h, int *info_retorno)
{
  int i, j, pos,
      info_mediano; //auxiliar para armazenar a chave que irá subir para o pai
  arvoreB *temp, *filho_dir; //ponteiro para o filho à direita da chave

  if (raiz == NULL)
   {
     //nó anterior é o ideal para inserir a nova chave
     *h = true;
     *info_retorno = info;
     return(NULL);
   }
  else {
        pos = busca_binaria(raiz,info);
        if (raiz->num_chaves > pos && raiz->chaves[pos] == info)
          {
            printf("Chave já contida na Árvore");
            *h = false;
          }
        else {
```

```
//percorrer a árvore até encontrar o nó folha para inserir a chave.
filho_dir = insere(raiz->filhos[pos],info,h,info_retorno);
if (*h) //Se true deve inserir a info_retorno no nó.
 {
    if (raiz->num_chaves < MAX_CHAVES) //Tem espaço na página
    {
      insere_chave(raiz, *info_retorno, filho_dir);
      *h = false;
    }
  else { //é necessário subdividir
        temp = (arvoreB *) malloc (sizeof(arvoreB));
        temp->num_chaves = 0;

       //inicializa filhos com NULL
       for (i = 0; i < MAX_FILHOS; i++)
         temp->filhos[i] = NULL;

       //elemento mediano que vai subir para o pai
       info_mediano = raiz->chaves[MIN_OCUP];
       //insere metade do nó raiz no temp
       temp->filhos[0] = raiz->filhos[MIN_OCUP+1];
       for (i = MIN_OCUP + 1; i < MAX_CHAVES; i++)
         insere_chave(temp, raiz->chaves[i], raiz->filhos[i+1]);
       //atualiza nó raiz.
       for (i = MIN_OCUP; i<MAX_CHAVES; i++)
       {
```

```
                raiz->chaves[i] = 0;
                raiz->filhos[i+1] = NULL;
              }
              raiz->num_chaves = MIN_OCUP;
              //verifica em qual nó será inserida a nova chave
              if (pos <= MIN_OCUP)
                  insere_chave(raiz, *info_retorno, filho_dir);
              else insere_chave(temp, *info_retorno, filho_dir);
            //retorna o mediano para inseri-lo no nó pai e o temp como filho
direito do mediano.          *info_retorno = info_mediano;
              return(temp);
            }
          }
        }
      }
    arvoreB *insere_arvoreB(arvoreB *raiz, int info)
    {
      bool h;
      int info_retorno, i;
      arvoreB *filho_dir, *nova_raiz;
      filho_dir = insere(raiz,info,&h,&info_retorno);
      if (h)
       { //aumetar a altura da árvore
         nova_raiz = (arvoreB *) malloc (sizeof(arvoreB));
         nova_raiz->num_chaves = 1;
```

```
    nova_raiz->chaves[0] = info_retorno;
    nova_raiz->filhos[0] = raiz;
    nova_raiz->filhos[1] = filho_dir;
    for (i = 2; i <= MAX_CHAVES; i++)
      nova_raiz->filhos[i] = NULL;
    return(nova_raiz);
  }
 else return(raiz);
}
```

2º exemplo: programa em C

```
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>
#define m 2
#define mm 4
typedef int Tipo_chave;
typedef struct
{
  Tipo_chave chave_arvore;
} Registro;

typedef struct Pagina_str *Apontador;
typedef struct Pagina_str
{
  int n;
```

```
  Registro r[mm];
  Apontador p[mm + 1];
} Pagina;
typedef Apontador TipoDicionario;
void Inicializa(TipoDicionario *Dicionario)
{
  *Dicionario = NULL;
}
void Pesquisar(Registro *x, Apontador Ap)
{
  int i;

  if (Ap == NULL)
  {
    printf("Erro: Registro nao esta presente.\n");
    system("PAUSE");
    return;
  }
  i = 1;
  while (i < Ap->n && x->chave_arvore > Ap->r[i - 1].chave_arvore)
    i++;
  if (x->chave_arvore == Ap->r[i - 1].chave_arvore)
  {
    *x = Ap->r[i - 1];
    return;
  }
```

```
  if (x->chave_arvore < Ap->r[i - 1].chave_arvore)
    Pesquisar(x, Ap->p[i - 1]);
  else
    Pesquisar(x, Ap->p[i]);
}
void Insere_pagina(Apontador Ap, Registro Reg, Apontador ApDir)
{
  int k;
  int nao_encontrou;

  k = Ap->n;
  nao_encontrou = k > 0;
  while (nao_encontrou)
  {
    if (Reg.chave_arvore >= Ap->r[k - 1].chave_arvore)
    {
      nao_encontrou = 0;
      break;
    }
    Ap->r[k] = Ap->r[k - 1];
    Ap->p[k + 1] = Ap->p[k];
    k--;
    if (k < 1)
      nao_encontrou = 0;
  }
  Ap->r[k] = Reg;
```

```
  Ap->p[k + 1] = ApDir;
  Ap->n++;
}
void Inserir(Registro Reg, Apontador Ap, int *Cresceu, Registro *RegRetorno,
Apontador *ApRetorno)
{
  Apontador ApTemp;
  int i, j;

  if (Ap == NULL)
  {
    *Cresceu = 1;
    *RegRetorno = Reg;
    *ApRetorno = NULL;
    return;
  }
  i = 1;
  while (i < Ap->n && Reg.chave_arvore > Ap->r[i - 1].chave_arvore)
    i++;
  if (Reg.chave_arvore == Ap->r[i - 1].chave_arvore)
  {
    printf(" Erro: Registro ja esta presente.\n");
    system("PAUSE");
    *Cresceu = 0;
    return;
  }
```

```
if (Reg.chave_arvore < Ap->r[i - 1].chave_arvore)
  Inserir(Reg, Ap->p[i - 1], Cresceu, RegRetorno, ApRetorno);
else
  Inserir(Reg, Ap->p[i], Cresceu, RegRetorno, ApRetorno);
if (!*Cresceu)
  return;
if (Ap->n < mm)
{
  Insere_pagina(Ap, *RegRetorno, *ApRetorno);
  *Cresceu = 0;
  return;
}
 ApTemp = (Apontador) malloc(sizeof(Pagina));
ApTemp->n = 0;
ApTemp->p[0] = NULL;
if (i <= m + 1)
{
  Insere_pagina(ApTemp, Ap->r[mm - 1], Ap->p[mm]);
  Ap->n--;
  Insere_pagina(Ap, *RegRetorno, *ApRetorno);
}
else
  Insere_pagina(ApTemp, *RegRetorno, *ApRetorno);
for (j = m + 2; j <= mm; j++)
  Insere_pagina(ApTemp, Ap->r[j - 1], Ap->p[j]);
Ap->n = m;
```

```
  ApTemp->p[0] = Ap->p[m + 1];
  *RegRetorno = Ap->r[m];
  *ApRetorno = ApTemp;
}
void Inserir_chave(Registro Reg, Apontador *Ap)
{
  int Cresceu;
  Registro RegRetorno;
  Apontador ApRetorno;
  Apontador ApTemp;
  Inserir(Reg, *Ap, &Cresceu, &RegRetorno, &ApRetorno);
  if (Cresceu) {
    ApTemp = (Apontador) malloc(sizeof(Pagina));
    ApTemp->n = 1;
    ApTemp->r[0] = RegRetorno;
    ApTemp->p[1] = ApRetorno;
    ApTemp->p[0] = *Ap;
    *Ap = ApTemp;
  }
}
void Reconstitui(Apontador ApPag, Apontador ApPai, int PosPai, int *Diminuiu)
{
  Apontador Aux;
  int DispAux, j;
  if (PosPai < ApPai->n) {
    Aux = ApPai->p[PosPai + 1];
```

```
DispAux = (Aux->n - m + 1) / 2;
ApPag->r[ApPag->n] = ApPai->r[PosPai];
ApPag->p[ApPag->n + 1] = Aux->p[0];
ApPag->n++;
if (DispAux > 0) {
  for (j = 1; j < DispAux; j++)
    Insere_pagina(ApPag, Aux->r[j - 1], Aux->p[j]);
  ApPai->r[PosPai] = Aux->r[DispAux - 1];
  Aux->n -= DispAux;
  for (j = 0; j < Aux->n; j++)
    Aux->r[j] = Aux->r[j + DispAux];
  for (j = 0; j <= Aux->n; j++)
    Aux->p[j] = Aux->p[j + DispAux];
  *Diminuiu = 0;
}
else
{
  for (j = 1; j <= m; j++)
    Insere_pagina(ApPag, Aux->r[j - 1], Aux->p[j]);
  free(Aux);
  for (j = PosPai + 1; j < ApPai->n; j++)
  {
    ApPai->r[j - 1] = ApPai->r[j];
    ApPai->p[j] = ApPai->p[j + 1];
  }
  ApPai->n--;
```

```
    if (ApPai->n >= m)
      *Diminuiu = 0;
  }
}
else
{
  Aux = ApPai->p[PosPai - 1];
  DispAux = (Aux->n - m + 1) / 2;
  for (j = ApPag->n; j >= 1; j--)
    ApPag->r[j] = ApPag->r[j - 1];
  ApPag->r[0] = ApPai->r[PosPai - 1];
  for (j = ApPag->n; j >= 0; j--)
    ApPag->p[j + 1] = ApPag->p[j];
  ApPag->n++;
  if (DispAux > 0) {
    for (j = 1; j < DispAux; j++)
      Insere_pagina(ApPag, Aux->r[Aux->n - j], Aux->p[Aux->n - j + 1]);
    ApPag->p[0] = Aux->p[Aux->n - DispAux + 1];
    ApPai->r[PosPai - 1] = Aux->r[Aux->n - DispAux];
    Aux->n -= DispAux;
    *Diminuiu = 0;
  }
  else
  {
    for (j = 1; j <= m; j++)
      Insere_pagina(Aux, ApPag->r[j - 1], ApPag->p[j]);
```

```
      free(ApPag);
      ApPai->n--;
      if (ApPai->n >= m)
        *Diminuiu = 0;
    }
  }
}

void Antecessor(Apontador Ap, int Ind, Apontador ApPai, int *Diminuiu)
{
  if (ApPai->p[ApPai->n] != NULL)
  {
    Antecessor(Ap, Ind, ApPai->p[ApPai->n], Diminuiu);
    if (*Diminuiu)
      Reconstitui(ApPai->p[ApPai->n], ApPai, ApPai->n, Diminuiu);
    return;
  }
  Ap->r[Ind - 1] = ApPai->r[ApPai->n - 1];
  ApPai->n--;
  *Diminuiu = ApPai->n < m;
}
void Retirar(Tipo_chave Ch, Apontador *Ap, int *Diminuiu)
{
  int Ind, j;
  Apontador WITH;
  if (*Ap == NULL)
```

```
{
  printf("Erro: registro nao esta na arvore.\n");
  system("PAUSE");
  *Diminuiu = 0;
  return;
}
WITH = *Ap;
Ind = 1;
while (Ind < WITH->n && Ch > WITH->r[Ind - 1].chave_arvore)
  Ind++;
if (Ch == WITH->r[Ind - 1].chave_arvore)
{
  if (WITH->p[Ind - 1] == NULL) {
    WITH->n--;
    *Diminuiu = WITH->n < m;
    for (j = Ind; j <= WITH->n; j++)
    {
      WITH->r[j - 1] = WITH->r[j];
      WITH->p[j] = WITH->p[j + 1];
    }
    return;
  }
  Antecessor(*Ap, Ind, WITH->p[Ind - 1], Diminuiu);
  if (*Diminuiu)
    Reconstitui(WITH->p[Ind - 1], *Ap, Ind - 1, Diminuiu);
  return;
```

```
  }
  if (Ch > WITH->r[Ind - 1].chave_arvore)
    Ind++;
  Retirar(Ch, &WITH->p[Ind - 1], Diminuiu);
  if (*Diminuiu)
    Reconstitui(WITH->p[Ind - 1], *Ap, Ind - 1, Diminuiu);
}
void Remover_chave(Tipo_chave Ch, Apontador *Ap)
{
  int Diminuiu;
  Apontador Aux;

  Retirar(Ch, Ap, &Diminuiu);
  if (Diminuiu && (*Ap)->n == 0) {
    Aux = *Ap;
    *Ap = Aux->p[0];
    free(Aux);
  }
}
void Listar_arvore(Apontador p, int Nivel)
{
  int i;

  if (p == NULL)
    return;
  for (i = 1; i <= Nivel; i++)
```

```
    printf("    ");
  for (i = 0; i < p->n; i++)
    printf("%4d", p->r[i].chave_arvore);
  putchar('\n');
  for (i = 0; i <= p->n; i++)
    Listar_arvore(p->p[i], Nivel + 1);
}
int main()
{
  Apontador *arv;
  Registro registro;
  char entrada;
  system("CLS");
  arv=(Apontador*) malloc(sizeof(Apontador));
  Inicializa(arv);
  system("CLS");
  printf("-----MENU-----\n");
  printf("--------------\n");
  while(1)
  {
    system("CLS");
    printf("-----MENU-----\n");
    printf("--------------\n");
    printf("1 - Inserir na arvore\n");
    printf("2 - Remover da arvore\n");
    printf("3 - Listar arvore\n");
```

```
printf("4 - Sair\n");
printf("\nDigite sua opcao: ");
scanf("%c", &entrada);
if (entrada=='4')
  break;
switch(entrada)
{
  case '1':
    do
    {
       system("CLS");
       printf("Inserir na arvore\n");
       printf("--------\n");
       printf("Digite o valor da chave a ser inserida: ( para finalizar)\n--> ");
       scanf("%d", &registro.chave_arvore);
       Inserir_chave(registro, arv);
    }while(registro.chave_arvore!=999);
    break;
  case '2':
    do
    {
       system("CLS");
       printf("Remover da arvore\n");
       printf("-------\n");
       printf("Digite o valor da chave a ser removida: (999 para finalizar)\n--> ");
       scanf("%d", &registro.chave_arvore);
```

```
            Remover_chave(registro.chave_arvore, arv);
        }while(registro.chave_arvore!=999);
        break;
      case '3':
        system("CLS");
        printf("Listar arvore\n");
        printf("---------\n");
        Listar_arvore(*arv, mm);
        system("PAUSE");
              break;
    }
    }
    system("PAUSE");
  }
```



ACOMPANHE NA WEB

Árvores B

As árvores B são árvores balanceadas projetadas para trabalhar com dispositivos de armazenamento secundário, como discos magnéticos. Elas visam otimizar as operações de entrada e saída nos dispositivos.

Disponível em: <http://www.lcad.icmc.usp.br/~nonato/ED/B_arvore/btree.htm>. Acesso em: set. 2014.

Árvores B

As árvores B são árvores balanceadas desenvolvidas para otimizar o acesso ao armazenamento secundário.

Disponível em: <http://www.ic.unicamp.br/~zanoni/mo637/aulas/arvoresB.pdf>.

Acesso em: set. 2014.

Árvores B-tree

As multidirecionais ou árvores B são desenvolvidas para otimizar o acesso ao armazenamento secundário.

Disponível em: <https://www.youtube.com/watch?v=T8Ki6tpCn9g>. Acesso em: set. 2014.

Tempo: 14:19



AGORA É A SUA VEZ

Instruções:

Agora, chegou a sua vez de exercitar seu aprendizado. A seguir, você encontrará algumas questões de múltipla escolha e dissertativas. Leia cuidadosamente os enunciados e atente-se para o que está sendo pedido.

Questão 1

Questão 1: Com o intuito de verificar as habilidades de programação adquiridas até agora, a atividade proposta é a solução de um problema simples, utilizando os recursos de programação estudados no tema anterior. Tais conhecimentos são extremamente necessários na sequência de estudo dos conteúdos, pois teremos muitos exemplos e exercícios baseados na linguagem C.

Problema proposto: Baseando-se na estrutura de programação da linguagem C, crie um programa que carregue uma árvore binária e resolva os itens abaixo:

Inserir um número na árvore.

Consultar um número da árvore.

Excluir um número da árvore.

```
#include <iostream>

#include <stdio.h>

struct arvore
```

```
{
    int num;
    arvore *direita, *esquerda;
};
arvore *raiz, *aux;
arvore *insere_R(arvore *aux, int num)
{
  if (aux == NULL)
   {
    aux = new (arvore);
    aux->num = num;
    aux->esquerda = NULL;
    aux->direita = NULL;
   }
  else if (num < aux->num)
            aux->esquerda=insere_R(aux->esquerda, num);
       else
            aux->direita=insere_R(aux->direita, num);
  return aux;
}
int consultar(arvore *aux, int num, int achou)
{
  if (aux != NULL && achou == 0)
  {
  if (aux->num == num)
    {
```

```
      achou = 1;
    }
  else if (num < aux->num)
          achou = consultar(aux->esquerda, num, achou);
       else
          achou = consultar(aux->direita, num, achou);
  }
  return achou;
}
arvore *remove(arvore *aux, int num)
{
  arvore *p, *p2;
  if (aux->num == num)
   {
    if (aux->esquerda == aux->direita)
      {
          delete(aux);
          return NULL;
      }
    else if (aux->esquerda == NULL)
            {
              p=aux->direita;
              delete(aux);
              return p;
             }
            else if (aux->direita == NULL)
```

```
                    {
                      p=aux->esquerda;
                      delete(aux);
                      return p;
                     }
                    else
                     {
                       p2= aux->direita;
                       p= aux->direita;
                       while (p->esquerda)
                        {
                          p=p->esquerda;
                        }
                        p->esquerda = aux->esquerda;
                        delete(aux);
                        return p2;
                     }
   }
   if (aux->num < num)
    aux->direita = remove(aux->direita, num);
   else
    aux->esquerda = remove(aux->esquerda, num);
   return aux;
}
int main()
{
```

```
int op, achou, numero;
raiz = NULL;
do
{
 system("CLS");
 printf("\nMenu de opções");
 printf("\n1 - Inserir um número na árvore");
 printf("\n2 - Consultar um número da árvore");
 printf("\n3 - Excluir um número da árvore");
 printf("\n4 - Sair");
 printf("\nDigite sua opção: ");
 scanf("%d", &op);
 if (op < 1 || op > 4)
  printf("\n\nOpção inválida.\n");
 if (op == 1)
  {
    std::cout << "\nDigite o número a ser inserido: ";
    scanf("%d", &numero);
    raiz=insere_R(raiz,numero);
    printf("\nNúmero inserido.\n");
  }
 if (op == 2)
  {
    std::cout << "\nDigite o número a ser consultado: ";
     scanf("%d", &numero);
    achou = consultar(raiz,numero,0);
```

```
      if (achou == 0)
          printf("\nNúmero não encontrado.\n");
      else
          printf("\nNúmero encontrado.\n");
    }
   if (op == 3)
    {
     if (raiz == NULL)
       printf("Árvore vazia.\n");
     else
      {
            std::cout << "\nDigite o número a ser removido: ";
             scanf("%d", &numero);
            achou = consultar(raiz,numero,0);
            if (achou == 0)
             printf("\nNúmero não encontrad.\n");
            else
             {
               raiz=remove(raiz,numero);
               printf("\nNúmero removido.\n");
             }
      }
    }
    system("PAUSE");
   }
   while (op!=4);
}
```

Questão 2

Assinale a alternativa incorreta com relação às propriedades de uma árvore B de ordem n.

a) Cada nó pode ter no máximo n filhos.

b) Cada nó, inclusive o nó raiz e os nós folhas, deve ter pelo menos n/2 filhos.

c) O nó raiz deve ter pelo menos dois filhos, a menos que seja um nó folha.

d) Todo nó não-folha com m filhos deverá ter n-1 chaves.

e) Nenhuma das alternativas é verdadeira.

Questão 3

Dentre as alternativas a seguir, qual satisfaz uma propriedade de uma árvore B de ordem n?

a) Todo nó não-folha com m filhos deverá ter n-1 chaves.

b) O nó folha deve ter pelo menos dois filhos, a menos que seja um nó raiz.

c) Cada nó, inclusive o nó raiz e os nós folhas, deve ter pelo menos n/2 filhos.

d) Cada nó pode ter no máximo n-1 filhos.

e) Nenhuma das alternativas é verdadeira.

Questão 4

Dada uma árvore B de ordem 20, qual é o número máximo de chaves que podem ser armazenadas por nó folha? Fundamente sua resposta.

Questão 5

Nas operações de inserção, quando a página é dividida, os nós são divididos igualmente entre as páginas nova e velha. Dada uma árvore B de ordem 20, qual é o número mínimo de chaves que um nó pode armazenar, exceto o nó raiz? Fundamente sua resposta



FINALIZANDO

Os conceitos e exemplos práticos em C apresentados neste tema deixaram bem claro que a estrutura de dados do tipo árvore multidirecional (árvore B) é muito versátil na solução de problemas do cotidiano.

Na estrutura do tipo árvore, cada elemento armazena um tipo de dado e de ponteiros para o elemento à esquerda e à direita, o que permite a inserção dos valores na árvore de forma recursiva.

Diferentemente das árvores de busca binária e AVL, a árvore multidirecional não precisa ser rebalanceada. Outra característica importante da árvore multidirecional, que aponta vantagens se comparada a outros tipos de árvores, é o menor tempo de acesso e pesquisa. Nas operações de inserção e remoção de dados, o nó não pode ser maior que sua ordem e, também, não pode ser menor que sua ordem dividida por 2.

As operações básicas de inserção, consulta e exclusão de um número da árvore foram abordadas com exemplos práticos.

REFERÊNCIAS

DUARTE, Luis; REIS, Kleiton Marques dos; SCHILING, Adler. Árvores B-tree. Vídeo/YouTube. Disponível em: <https://www.youtube.com/watch?v=T8Ki6tpCn9g>. Acesso em: 25 jun. 2014.

IC UNICAMP. Árvores B. Complexidade de Algoritmos II. Campinas: UNICAMP; Instituto de Computação, 14 set. 2007. Disponível em: <http://www.ic.unicamp.br/~zanoni/mo637/aulas/arvoresB.pdf>. Acesso em: 25 jun. 2014.

LCAD. Árvores B. São Paulo: USP; Laboratório de Computação de Alto Desempenho. Disponível em: <http://www.lcad.icmc.usp.br/~nonato/ED/B_arvore/btree.htm>. Acesso em: 25 jun. 2014.

TENENBAUM, Aaron M.; LANGSAM, Yedidyah; AUGENSTEIN, Moshe J. Estruturas de Dados Usando C. São Paulo: Makron Books, 1995.

GLOSSÁRIO

Banco de dados: É uma estrutura utilizada para armazenamento, que compreende um conjunto de registros cujo objetivo é organizar e armazenar dados e informações.

Sistema de arquivos: São rotinas e estruturas lógicas que auxiliam os sistemas operacionais no controle de acesso ao disco rígido. O volume de arquivos e acessos tende a crescer junto com a capacidade de armazenamento dos discos, e isso exige que os sistemas de arquivos sejam mais robustos para acompanharem o volume de acessos aos arquivos.

Tabela hash: Também é conhecida como tabela de dispersão ou de espalhamento. É um tipo de estrutura de dados que tem como principal característica a associação de chaves de pesquisa aos valores armazenados. Sua estrutura de busca tende a ser rápida, pois parte de uma chave simples para encontrar o valor buscado.

Grafos e suas aplicações

POR DENTRO DO TEMA

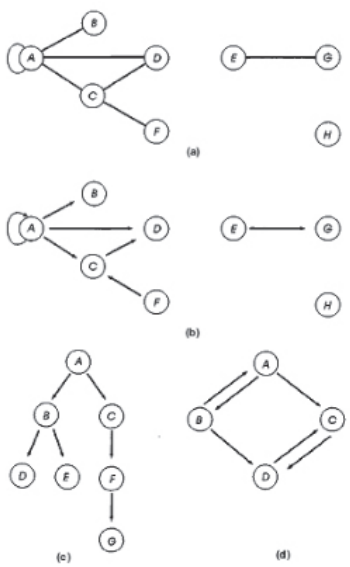
Uma definição simples de grafo seria a seguinte: um conjunto de vértices (ou nós) e um conjunto de **arestas** (ou arcos), sendo que cada aresta em um grafo é representada por um par de vértices. Um grafo G=(V,A) consiste em:

1. Conjunto finito de **vértices** V. Tais elementos de V são chamados vértices do grafo G.

2. Conjunto finito A de pares não ordenados de V. Tais elementos de A são chamados arestas do grafo G.

A Figura 8.1 a seguir apresenta vários exemplos de grafos, identificados pelas letras a, b, c e d:

Figura 8.1: Exemplos de grafos: (a), (b), (c) e (d)



Fonte: TENENBAUM; LANGSAM; AUGENSTEIN (1995).

> A sequência de nós é {A,B,C,D,E,F,G,H}, e o conjunto de arcos é {(A,B), (A,D), (A,C), (C,D), (C,F), (E,G), (A,A)}. Se os pares de nós que formam os arcos forem pares ordenados, diz-se que o grafo é um grafo orientado (ou dígrafo). As Figuras 1b, c e d ilustram três dígrafos. As setas entre os nós representam arcos. A ponta de cada seta representa o segundo nó no par ordenado de nós que forma um arco, e o final de cada seta representa o primeiro nó no par. O conjunto de arcos do grafo da Figura 1b é {<A,B>, <A,C>, <A,D>, <C,D>, <F,C>, <E,G>, <A,A>}. Observe que um grafo não precisa ser uma árvore (Figuras 1a, b e d), mas uma árvore tem de ser um grafo (Figura 1c). Observe também que um nó não precisa ter arcos associados a ele (nó H nas Figuras 1a e b). (TENENBAUM; LANGSAM; AUGENSTEIN, 1995)

Dizemos que um nó x qualquer incide no arco y somente se x representar no par ordenado de nós que forma y um dos dois nós. Neste caso, podemos dizer que y incide em x. O número de arcos que incidem em um nó determina o seu grau. Pode-se observar também que:

3. O número de arcos que um nó tem como cabeça representa seu grau de entrada.

4. O número de arcos que um nó tem como terminação de seta representa seu grau de saída.

Como exemplo, temos o nó A na Figura 8.1 (d), que tem grau 3, sendo grau de entrada 1 e grau de saída 2.

> Um caminho de comprimento k do nó a ao nó b é definido como uma sequência de k + 1 nós n1 , n2 ,..., nk , tal que n = a, nk = b e adjacent(ni + i) é true para todo i entre 1 e k. Se, para algum inteiro k, existir um caminho de comprimento k entre a e b, existirá um caminho de a até b. Um caminho de um nó para si mesmo é chamado ciclo. Se um grafo contiver um ciclo, ele será cíclico; caso contrário, será acíclico. Um grafo acíclico orientado é chamado dag, uma aglutinação das iniciais de directed acyclic graph. (TENENBAUM; LANGSAM; AUGENSTEIN, 1995).

Aplicações com Grafos

Busca em profundidade

Nos algoritmos de busca em profundidade, as arestas são percorridas partindo do vértice v que tenha sido descoberto por último e que tenha arestas não descobertas a partir dele.

Depois que todas as arestas partindo de v forem descobertas, uma nova busca se inicia para percorrer as arestas partindo do vértice a partir do qual v foi descoberto. Um busca em profundidade poderá gerar algumas árvores, dependendo do número de origens a partir das quais a busca é repetida.

Na busca em profundidade, assim como na busca em largura, os vértices são coloridos indicando seu estado.

Exemplo 1 - Busca em profundidade

```
#include <iostream>
#include <map>
#include <list>
#include <string>
using namespace std;
const string P = "preto";
const string B = "branco";
const string C = "cinza";

typedef list<int> listaAdjacencias;
typedef map<int, listaAdjacencias> grafo;
map<int, string> cor;
list<int> ordemTopologica;
void visitarNodo(grafo G, int u)
{
    cor[u] = C;
```

```
	for(listaAdjacencias::iterator i = G[u].begin(); i != G[u].end(); i++)
	{
		if ( cor[ *i ] == B )
			visitarNodo(G, *i);
	}
	cor[u] = P;
	ordemTopologica.push_back(u);
}
void buscaProfundidade(grafo G)
{
	for(grafo::iterator i = G.begin(); i != G.end(); ++i)
		cor[i->first] = B;

	for(grafo::iterator i = G.begin(); i != G.end(); ++i)
	{
		if ( cor[i->first] == B )
		{
			visitarNodo(G, i->first);
		}
	}
}
int temCiclo(grafo G)
{
	list<int>::iterator i, j;
```

```
        buscaProfundidade(G);

        for(i = ordemTopologica.begin(); i != ordemTopologica.end(); i++)
        {
                j = i;
                for(++j; j != ordemTopologica.end(); j++)
                        if (  find(G[*i].begin(), G[*i].end(), *j) != G[*i].end()  )
                                return(0);
        }
        return(1);
}

int main(int argc, char *argv[])
{
        FILE *arquivo;

        grafo G;
        int no, aresta;

        if( (arquivo = fopen ("c:\\grafos\\arquivo.txt", "r")) == NULL )
        {
                printf("\n\n Erro na leitura do arquivo.\n");
                exit(1);
        }

        while( !feof(arquivo) )
```

```
        {
                fscanf(arquivo,"%d %d", &no, &aresta);

                if ( G[no].empty() )
                        G[no] = listaAdjacencias();
                if ( G[aresta].empty() )
                        G[aresta] = listaAdjacencias();
                G[no].push_back(aresta);
        }
        fclose(arquivo);

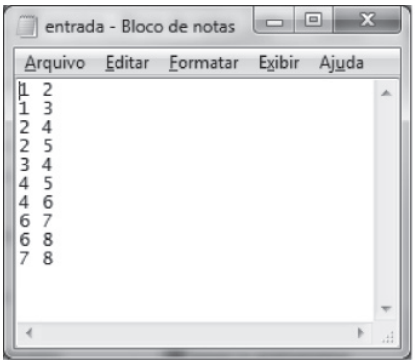
        for(grafo::iterator i = G.begin(); i != G.end(); ++i)
                if ( ! (i->second).empty() )
                        (i->second).sort();

        if ( temCiclo(G) == 1 )
                printf("Grafo aciclico.\n");
        else
                printf("Ha ciclos no grafo.\n");

        system("PAUSE");
    return(0);
}
```

Arquivo para teste – arquivo.txt

Antes de executar o programa anteriormente descrito, deve-se criar o arquivo a seguir na pasta c:\grafos\arquivo.txt.



Lista de adjacências

Uma lista de adjacências em grafos pode ser definida como a representação de todas as arestas ou arcos de um grafo em uma lista.

Em grafos não direcionados, as entradas são conjuntos de dois nós contendo as extremidades da aresta correspondente. No caso de grafos dirigidos, a entrada é uma tupla de dois nós, representando origem e destino de um arco qualquer. Na maioria das representações, as listas de adjacências não são ordenadas.

Exemplo 2 - Lista de adjacências

```
#include <iostream>

#include <stdio.h>

#define TOT_VERT 8


typedef struct itens{

    int cmp;

    struct itens* prox;

}itens;


itens lista[TOT_VERT+1];


void Listar(itens *lista);

void Insere(itens *lista, int a, int b);
```

```
int main(int argc, char *argv[]){
  int i,a,b;

  FILE *fp;
  fp = fopen("c:\\grafos\\entrada.txt","r");
  if (!fp) {
    printf("Erro na tentativa de abrir o arquivo %s.\n");
    return 0;
  }

  for(i=1; i<=TOT_VERT; i++){
    lista[i].cmp = 0;
    lista[i].prox = NULL;
  }

  fscanf(fp,"%d %d", &a, &b);
  while (!feof(fp)) {
    Insere(lista,a,b);
    Insere(lista,b,a);
    fscanf(fp,"%d%d", &a, &b);
  }

  Listar(lista);
  system("PAUSE");
}
```

```
void Listar(itens *lista){
  int i;
  itens * Temporaria;
  for(i=1; i<=TOT_VERT; i++) {
    Temporaria = lista[i].prox;
    printf("%2d: (%d) ==>", i, lista[i].cmp);
    while (Temporaria != NULL) {
      printf("%d  ", Temporaria->cmp);
      Temporaria = Temporaria->prox;
    }
    printf("\n");
  }
}
void Insere(itens *lista, int a, int b){
  itens *aux;
  itens *Temporaria;

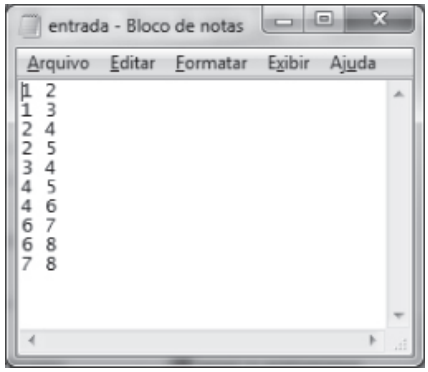
  aux = (itens*) malloc((int)sizeof(itens));
  aux->cmp = b;
  aux->prox = NULL;

  lista[a].cmp++;
  if(lista[a].prox == NULL)
    lista[a].prox = aux;
  else          {
```

```
    Temporaria = lista[a].prox;
    if (Temporaria->cmp > b) {
      aux->prox = Temporaria;
      lista[a].prox = aux;
    }
    else if (Temporaria->prox == NULL) {
      aux->prox = Temporaria->prox;
      Temporaria->prox = aux;
    }
    else {
      while((Temporaria->prox != NULL) &&(Temporaria->prox->cmp < b))
        Temporaria = Temporaria->prox;
      aux->prox = Temporaria->prox;
      Temporaria->prox = aux;
    }
  }
}
```

Arquivo para teste – arquivo.txt

Antes de executar o programa anteriormente descrito, deve-se criar o arquivo a seguir na pasta c:\grafos\ entrada.txt.



Menor caminho

> Num grafo ponderado, ou rede, deseja-se frequentemente achar o menor caminho entre dois nós, s e t. O menor caminho é definido como um caminho de s até t, de modo que a soma dos pesos dos arcos do caminho seja minimizada. Para representar a rede, presumimos uma função de peso, de modo que weight(i,j) seja o peso do arco de i a j. Se não existir um arco de i até j, weight(i,j) será definida com valor arbitrariamente grande para indicar o custo infinito (ou seja, a impossibilidade) de seguir diretamente de i até j. (TENENBAUM; LANGSAM; AUGENSTEIN, 1995).

Um dos algoritmos mais utilizados para calcular o caminho de custo mínimo entre vértices de um grafo é o Algoritmo de Dijkstra. Sua execução parte da escolha de um vértice para ser a raiz da busca. Partindo do vértice escolhido, o algoritmo calcula o custo mínimo entre este vértice e os outros vértices do grafo. Apesar de ser um algoritmo simples, apresenta um bom desempenho na execução. Entretanto, caso existam arcos com valores negativos, o algoritmo não garante exatidão da solução.

Na busca do custo mínimo, o algoritmo parte de uma estimativa inicial e vai ajustando esta estimativa.

Exemplo 3 – Menor caminho

```
#include<stdio.h>

#include<stdlib.h>

#define DIM 7

//Variáveis globais

int Grafo[DIM][DIM],Grafo1[DIM][DIM],M_c[DIM][DIM],M_p[DIM][DIM],Conectividade[DIM][DIM],op,ori,des;

void CarreGrafoar(int m[DIM][DIM], int v)

{

  int i,j;

  i=j=0;

    while(i!=7)
```

```
        {
            while(j!=7)
              {
                m[i][j]=v;
                j++;
              }
          j=0;
          i++;
        }
}
void Mostrar(int m[DIM][DIM])
{
   int i,j,k;
   system("CLS");
   i=j=0;
   k=1;
   printf("       ");
     while(k!=8)
        {
          printf("  %d  ",k);
          k++;
        }
   printf("  \n\n\n  ");
   k=1;
     while(i!=7)
        {
```

```
            printf("  %d  ",k);
              while(j!=7)
                 {
                    printf("  %d  ",m[i][j]);
                    j++;
                 }
            i++;
            j=0;
            k++;
            printf("  \n\n  ");
         }
}
void Mostrar_1(int m[DIM][DIM])
{
   int i,j,k;
   i=j=0;
   k=1;
   system("CLS");
   printf("    ");
     while(k!=8)
        {
           printf("%d   ",k);
           k++;
        }
   k=1;
   printf("\n\n\n");
```

```
    while(i!=7)
      {
        printf("%d   ",k);
          while(j!=7)
            {
              printf("%d   ",(m[i][j]+1));
              j++;
            }
        i++;
        j=0;
        k++;
        printf("\n\n");
      }
}
void M_conectar(int m[DIM][DIM], int a, int b, int v)
{
  m[a-1][b-1]=v;
}

int M_adjacente(int m[DIM][DIM], int a, int b)
{
  if(m[a-1][b-1] == 0)
    {
      return(0);
    }
  else
```

```
    {
      return(1);
    }
}
void M_custo(int Grafo[DIM][DIM], int c[DIM][DIM], int p[DIM][DIM])
{
  int i,j,k;
  i=j=k=0;
    while(i!=7)
      {
          while(j!=7)
            {
                if(Grafo[i][j]!=0)
                  {
                    c[i][j]=Grafo[i][j];
                  }
                else
                  {
                    c[i][j]=99;
                  }
              j++;
            }
        i++;
        j=0;
      }
  i=0;
```

```
  while(i!=7)
    {
      c[i][i]=0;
      i++;
    }
i=0;
j=k=1;
  while(k!=7)
    {
        while(i!=7)
          {
              while(j!=7)
                {
                    if((c[i][k] + c[k][j]) < c[i][j])
                      {
                        c[i][j] = c[i][k] + c[k][j];
                        p[i][j] = k;
                      }
                  j++;
                }
            i++;
            j=0;
          }
      k++;
      i=0;
    }
```

```
}
void Mostrar_caminho(int m[DIM][DIM], int a, int b)
{
  int k,d,r;
  k=((m[a][b])+1);
    if(k!=0)
      {
        Mostrar_caminho(m, a, k-1);
        printf("%d => ",k);
        Mostrar_caminho(m, k-1, b);
      }
}
void M_conectividade(int m[DIM][DIM], int cd[DIM][DIM])
{
  int i,j,k;
    for(i=0;i<=6;i++)
      {
          for(j=0;j<=6;j++)
            {
              cd[i][j]=m[i][j];
            }
      }
    for(k=0;k<=6;++k)
      {
          for(i=0;i<=6;++i)
            {
```

```
                for(j=0;j<=6;++j)
                  {
                      if(cd[i][j]==0)
                        {
                          cd[i][j]=((cd[i][k]) * (cd[k][j]));
                        }
                  }
            }
        }
}
int Mostrar_centro(int m[DIM][DIM])
{
   int i,j,x,vetor[DIM];
   i=0;
   j=0;
      while(i!=7)
        {
          vetor[i]=0;
          i++;
        }
   i=0;
      while(j!=7)
        {
            while(i!=7)
              {
                  if((m[i][j]) > vetor[j])
```

```
                    {
                        vetor[j]=m[i][j];
                    }
                i++;
            }
        j++;
        i=0;
      }
    x=99;
    i=0;
      while(i!=7)
        {
            if(vetor[i] < x)
              {
                x=vetor[i];
              }
        i++;
        }
    return(x+1);
  }
  void Montar_menu(int Grafo[DIM][DIM], int M_c[DIM][DIM], int M_p[DIM][DIM],
int Conectividade[DIM][DIM])
  {
    do{
      system("CLS");
      printf("1- Mostrar a matriz de adjacencia do grafo.\n");
```

```
printf("2- Mostrar a matriz de menores custos do grafo.\n");
printf("3- Mostrar a matriz de conectividade do grafo.\n");
printf("4- Mostrar o caminho\n");
printf("5- Sair");
printf("\nDigite a sua op: ");
scanf("%d",&op);
}while((op<1) || (op>7));
if(op==1)
{
    Mostrar(Grafo);
    system("PAUSE");
    Montar_menu(Grafo,M_c,M_p,Conectividade);
}
if(op==2)
{
    Mostrar(M_c);
    system("PAUSE");
    Montar_menu(Grafo,M_c,M_p,Conectividade);
}
if(op==3)
{
    Mostrar(Conectividade);
    system("PAUSE");
    Montar_menu(Grafo,M_c,M_p,Conectividade);
}
if(op==4)
```

```
{
    printf("Digite o vertice de origem: ");
    scanf("%d",&ori);
    while((ori<=0)||(ori>7))
    {
        printf("\nVertice invalido. \nDigite novamente.\n");
        printf("\nDigite o vertice de origem: ");
        scanf("%d",&ori);
    }
    printf("Digite o vertice de destino: ");
    scanf("%d",&des);
    while((des<=0)||(des>7))
    {
        printf("\nVertice invalido. \nDigite novamente.\n");
        printf("Digite o vertice de destino: ");
        scanf("%d\n\n",&des);
    }
    printf("%d => ",ori);
    Mostrar_caminho(M_p,ori-1,des-1);
    printf("%d\n\n",des);
    system("PAUSE");
    Montar_menu(Grafo,M_c,M_p,Conectividade);
}
if(op==5)
  {
    exit(1);
```

```
        }
}
void Inicializar_grafo()
{
   M_conectar(Grafo,1,2,1);
   M_conectar(Grafo,1,5,1);
   M_conectar(Grafo,1,7,9);
   M_conectar(Grafo,2,3,1);
   M_conectar(Grafo,2,4,4);
   M_conectar(Grafo,2,6,5);
   M_conectar(Grafo,3,4,2);
   M_conectar(Grafo,4,7,3);
   M_conectar(Grafo,5,4,10);
   M_conectar(Grafo,5,6,2);
   M_conectar(Grafo,6,3,8);
   M_conectar(Grafo,6,7,3);
   M_conectar(Grafo,7,2,6);
   M_conectar(Grafo1,1,2,1);
   M_conectar(Grafo1,1,5,1);
   M_conectar(Grafo1,1,7,1);
   M_conectar(Grafo1,2,3,1);
   M_conectar(Grafo1,2,4,1);
   M_conectar(Grafo1,2,6,1);
   M_conectar(Grafo1,3,4,1);
   M_conectar(Grafo1,4,7,1);
   M_conectar(Grafo1,5,4,1);
```

```
    M_conectar(Grafo1,5,6,1);
    M_conectar(Grafo1,6,3,1);
    M_conectar(Grafo1,6,7,1);
    M_conectar(Grafo1,7,2,1);
}
int main ()
{
    system("CLS");
    CarreGrafoar(Grafo,0);
    CarreGrafoar(M_c,0);
    CarreGrafoar(M_p,-1);
    CarreGrafoar(Grafo1,0);
    CarreGrafoar(Conectividade,0);
    Inicializar_grafo();
    M_custo(Grafo,M_c,M_p);
    M_conectividade(Grafo1,Conectividade);
    Montar_menu(Grafo,M_c,M_p,Conectividade);
}
```



ACOMPANHE NA WEB

Grafos I: Conceitos e Aplicações

Introdução aos grafos: definição, terminologia, algumas propriedades e exemplos de aplicações de grafos.

Disponível em: <http://wiki.icmc.usp.br/images/5/59/Grafos_I.pdf> Acesso em: 12 ago. 2014.

Algoritmos em Grafos

Suponha que existam seis sistemas computacionais (A, B, C, D, E e F) interconectados entre si. Esta informação pode ser representada por um diagrama chamado de grafo.

Disponível em: <http://homepages.dcc.ufmg.br/~loureiro/alg/052/pa2e_cap7.pdf> Acesso em: 12 ago. 2014.

Grafos - Estrutura de Dados 2

Grafos é um tipo abstrato de estrutura de dados em que, dado um número de objetos, há um conjunto de conexões entre pares destes objetos.

Disponível em: <http://www.youtube.com/watch?v=_uk1E-h63Kk> Acesso em: 12 ago. 2014.

Tempo: 5:18.



AGORA É A SUA VEZ

Instruções:

Agora, chegou a sua vez de exercitar seu aprendizado. A seguir, você encontrará algumas questões de múltipla escolha e dissertativas. Leia cuidadosamente os enunciados e atente-se para o que está sendo pedido.

Questão 1

Dada uma árvore B de ordem 35, qual é o número máximo de chaves que podem ser armazenadas por nó folha e o número mínimo de chaves que um nó pode armazenar, exceto o nó raiz? Fundamente sua resposta.

Questão 2

Dentre as teorias que envolvem o estudo dos grafos, uma diz que o número de arcos que incidem em um nó determina o seu grau. Pode-se observar também que:

a) O número de arcos que um nó tem como cabeça representa seu grau de entrada.

b) O número de arcos que um nó tem como terminação de seta representa seu

grau de entrada.

c) O número de arcos que um nó tem como cabeça representa seu grau de desvio padrão.

d) O número de arcos que um nó tem como terminação de seta representa seu grau de desvio padrão.

e) Nenhuma das alternativas é verdadeira.

Questão 3

Algumas estruturas de dados e técnicas de programação podem ser utilizadas nas implementações de soluções para problemas que envolvem as teorias dos grafos. Assinale a alternativa que corresponde a uma estrutura que é amplamente utilizada para implementar grafos:

a) Vetor unidimensional.

b) Matriz unidimensional.

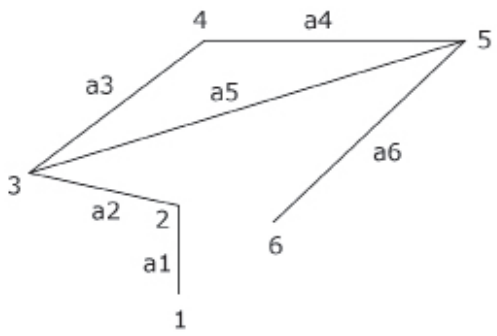
c) Matriz de adjacência.

d) Vetor.

e) Nenhuma das alternativas é verdadeira.

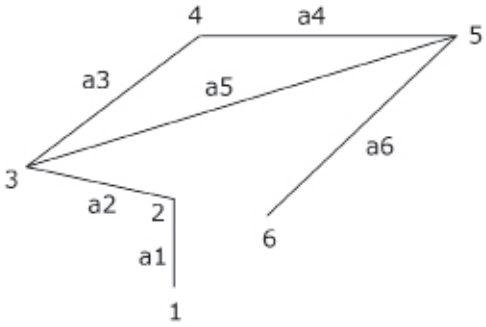
Questão 4

Observe o grafo a seguir. É correto afirmar que existem dois caminhos para percorrer do nó 3 para o nó 6? Se a resposta for sim, descreva os caminhos.



Questão 5

Observe o grafo a seguir. Existe algum ciclo no grafo? Se a resposta for sim, descreva os caminhos percorridos pelos ciclos.


4
a4
5
a3
a5
a6
3
a2
2
a1
6
1



FINALIZANDO

Os conceitos e exemplos práticos em C apresentados neste tema deixaram bem claro que os grafos são muito versáteis na solução de problemas do cotidiano.

Diferentemente de outros ramos da matemática, que muitas vezes apresentam trabalhos com base somente teórica, a teoria dos grafos é focada em problemas práticos do cotidiano. Seu embasamento está direcionado ao estudo de objetos combinatórios (grafos), que representam modelos para problemas em diversos ramos da matemática, computação, engenharia, indústria, entre outros.

Foram apresentados conceitos básicos sobre grafos e seu uso em C com alguns exemplos. Ao finalizar, tivemos alguns exercícios para reforçar o conteúdo estudado referente ao estudo dos grafos.



REFERÊNCIAS

CAMPELLO, Ricardo J. G. B. Grafos I: Conceitos e Aplicações. Estruturas de Dados. Disponível em: <http://wiki.icmc.usp.br/images/5/59/Grafos_I.pdf>. Acesso em: 28 jun. 2014.

LUOREIRO, Antonio Alfredo Ferreira. Algoritmos e Estruturas de Dados II: Algoritmos em Grafos. UFMG. Disponível em: <http://homepages.dcc.ufmg.br/~loureiro/alg/052/pa2e_cap7.pdf>. Acesso em: 28 jun. 2014.

SILVA, Luciano da; SILVEIRA, Artur Rafael da. Grafos: Estrutura de Dados 2. Vídeo/YouTube. Disponível em: <http://www.youtube.com/watch?v=_uk1E-h63Kk>. Acesso em: 28 jun. 2014.

TENENBAUM, Aaron M.; LANGSAM, Yedidyah; AUGENSTEIN, Moshe J. Estruturas de Dados usando C. São Paulo: Makron Books, 1995.



GLOSSÁRIO

Estrutura de dados: na computação, estrutura de dados é uma forma específica de organização e armazenamento de dados, para que sejam utilizados de forma eficaz. As estruturas de dados e seus algoritmos são muito utilizados na Ciência da Computação, em diversas áreas do conhecimento e com as mais diferentes finalidades na solução de problemas computacionais.

Aresta: na geometria, o encontro de duas faces resulta em linhas chamadas de arestas. A aresta também é chamada de "reta".

Vértice: é o ponto comum a duas ou mais retas, também definido como ponto de ligação entre as arestas. Os vértices são geralmente representados por um ponto.



unopar

ISBN 978-85-8482-572-1

